Department Artificial Intelligence and Data Science

Database Management System (PCC-251- AID)

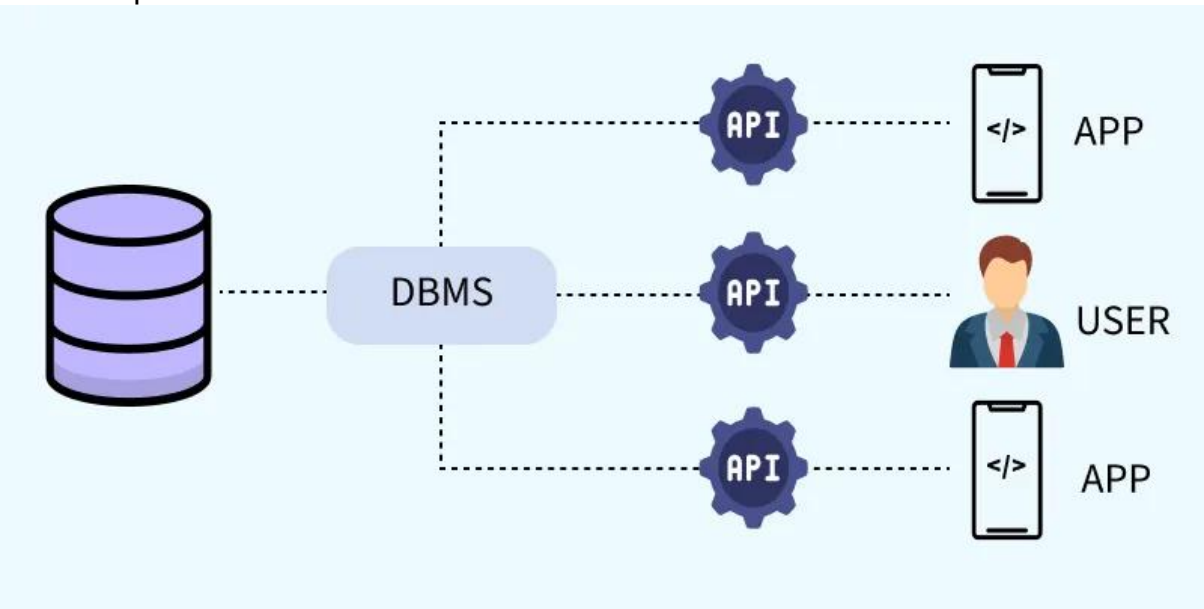
Unit – 1

Foundations of Data Management and ER Model

Introduction-

DBMS (Database Management System) is a software system that manages, stores, and retrieves data efficiently in a structured format.

- It allows users to create, update, and query databases efficiently.
- Ensures data integrity, consistency, and security across multiple users and applications.
- Reduces data redundancy and inconsistency through centralized control.
- Supports concurrent data access, transaction management, and automatic backups



DBMS acts as a bridge between a central database and multiple clients-including apps and users. It uses **APIs** to handle data requests, enabling apps and users to interact with the database securely and efficiently without directly accessing the data.

Problems with Traditional File-Based Systems

Before the introduction of modern DBMS, data was managed using basic file systems on hard drives. While this approach allowed users to store, retrieve and update files as needed, it came with numerous challenges

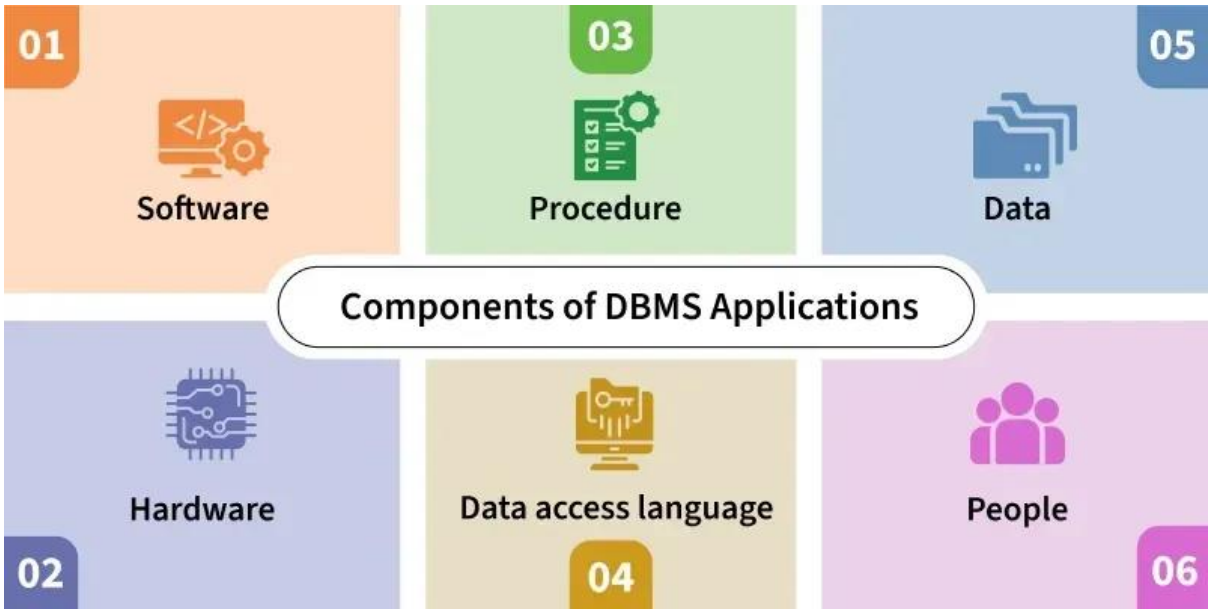
- **Data Redundancy:** Duplicate entries across files
- **Inconsistency:** Conflicting or outdated information

- **Difficult Access:** Manual file search required
- **Poor Security:** No control over data access
- **Single-User Access:** No support for collaboration
- **No Backup/Recovery:** Data loss was often permanent

A university file-based system storing data in separate files (e.g., Academics, Results, Hostels) often faced these problems.

Components of DBMS Applications

Any DBMS based applications are made up of six key components that work together to handle data effectively.



Components of DBMS Applications

1. Hardware

- Physical devices like servers, disks, input-output devices (keyboard, monitor, printer).
- Stores and processes data; interfaces between real-world inputs and digital systems.
- Examples: Personal computer hard disk, RAM, network devices used for DBMS operations.

2. Software

- Actual DBMS software like MySQL, Oracle, PostgreSQL.
- Includes the database engine, OS, network software, and application tools.
- Translates database access languages into operations.

3. Data

- Raw facts stored in structured or unstructured formats.
- **Operational Data**: Actual user data (e.g., name, age).
- **Metadata**: Data about data (e.g., storage time, size, data type).
- Core reason DBMS exists—to manage and store data efficiently.

4. Procedures

- Instructions and rules for using DBMS effectively.

- Covers setup, login/logout, data validation, backup, access control, and report generation.
- Helps ensure consistent and secure use of the system.

5. Database Access Language

- Used to interact with the database (create, read, update, delete data).
- Examples: SQL, MyAccess, Oracle PL/SQL.
- **DDL (Data Definition Language)** – CREATE, ALTER, DROP
- **DML (Data Manipulation Language)** – INSERT, UPDATE, DELETE

6. People

Users interacting with DBMS at different levels:

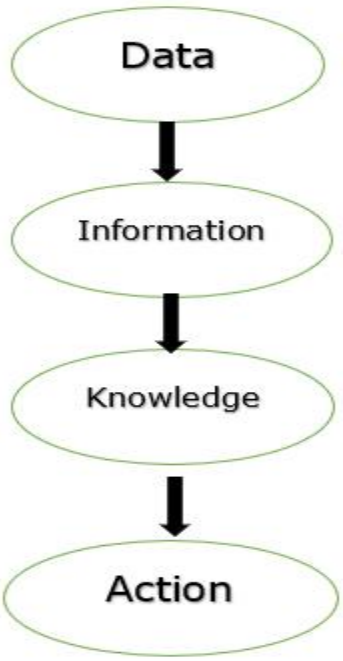
- **Database Administrators (DBA)** – Manage security, performance, user access.
- **Developers** – Build applications using the database.
- **End Users** – Use applications to access the database (e.g., students, employees).

Purpose of DBMS-

The purpose of DBMS is to transform the following ?

- Data into information.
- Information into knowledge.
- Knowledge to the action.

The diagram given below explains the process as to how the transformation of data to information to knowledge to action happens respectively in the DBMS ?



Previously, the database applications were built directly on top of the file system.

Drawbacks in File System

There are so many drawbacks in using the file system. These are mentioned below ?

- Data redundancy and inconsistency: Different file formats, duplication of information in different files.
- Difficulty in accessing data: To carry out new task we need to write a new program.
- Data Isolation ? Different files and formats.
- Integrity problems.
- Atomicity of updates ? Failures leave the database in an inconsistent state. For example, the fund transfer from one account to another may be incomplete.
- Concurrent access by multiple users.
- Security problems.

Database system offer so many solutions to all these problems

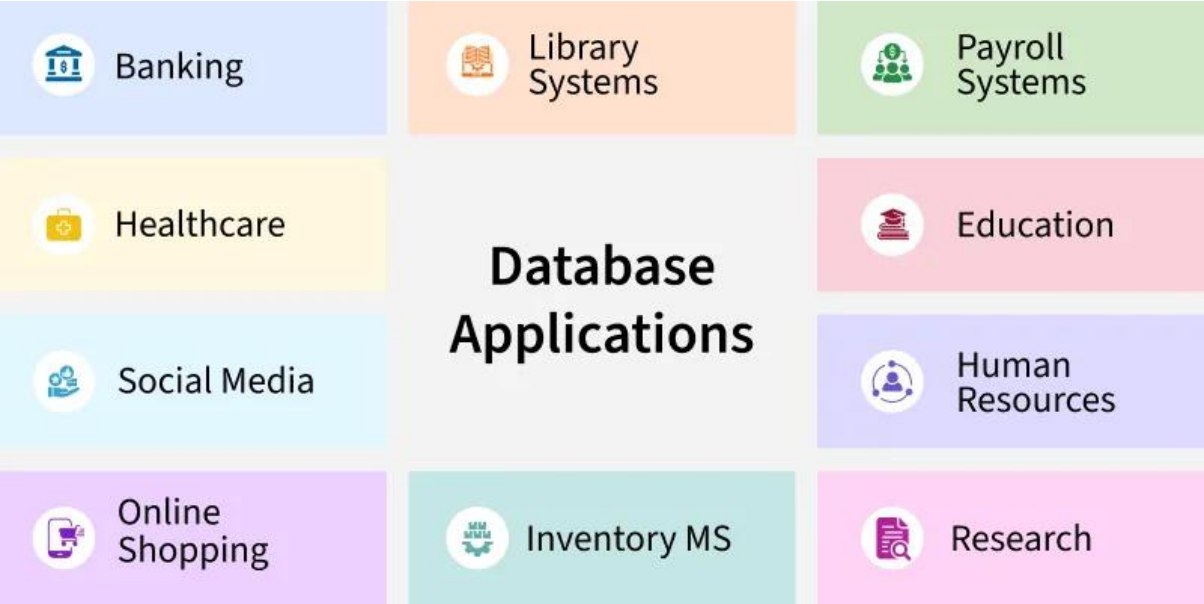
Uses of DBMS

The main uses of DBMS are as follows ?

- Data independence and efficient access of data.
- Application Development time reduces.
- Security and data integrity.
- Uniform data administration.
- Concurrent access and recovery from crashes.

Feature	File System	Database Management System (DBMS)
Data Organization	Stores data in files, organized based on file names and directories.	Stores data in tables, organized based on rows and columns.
Data Relationships	Limited ability to represent relationships between data items.	Can represent complex relationships between data items using primary and foreign keys.
Data Integrity	Provides limited data integrity features.	Offers robust data integrity features like referential integrity, data validation, and concurrency control.
Data Security	Provides basic security features like access control and permissions.	Provides advanced security features like encryption, role-based access control, and auditing.
Data Independence	Limited data independence, as data is tightly coupled with application programs.	High degree of data independence, as data is separated from application programs.
Data Consistency	May lead to data inconsistencies due to multiple copies of data.	Ensures data consistency through techniques like transaction processing and ACID properties.

Applications of DBMS



- **Banking**: Manages accounts and transactions.
- **E-commerce**: Tracks products, orders, and customers.
- **Healthcare**: Stores patient records and diagnoses.
- **Education**: Handles student grades and schedules.
- **Social Media**: Manages user profiles and interactions.
- **Data Science**: Supports analytics and predictions.

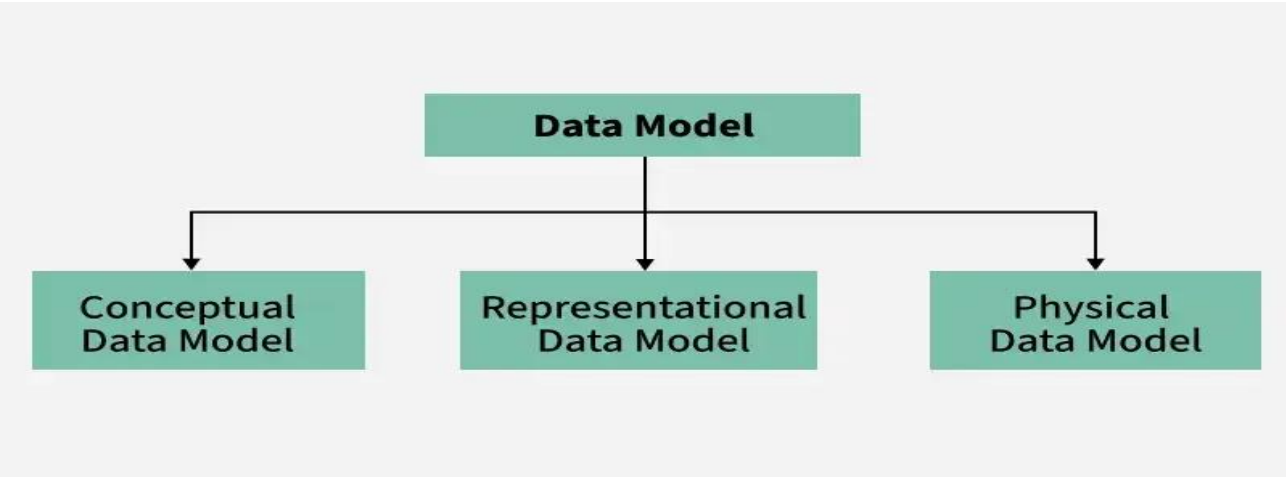
Data Models-

A Data Model in Database Management System (DBMS) is the concept of tools that are developed to summarize the description of the database. Data Models provide us with a transparent picture of data which helps us in creating an actual database. It shows us from the design of the data to its proper implementation of data.

Types of Relational Models

1. Conceptual Data Model
2. Representational Data Model
3. Physical Data Model

It is basically classified into 3 types:-



Data Models

1. **Conceptual Data Model**

The conceptual data model describes the database at a very high level and is useful to understand the needs or requirements of the database. It is this model, that is used in the requirement-gathering process i.e. before the Database Designers start making a particular database. One such popular model is the entity/relationship model (ER model). The E/R model specializes in entities, relationships, and even attributes that are used by database designers. In terms of this concept, a discussion can be made even with non-computer science(non-technical) users and stakeholders, and their requirements can be understood.

.

Characteristics of a conceptual data model

- Offers Organization-wide coverage of the business concepts.
- This type of Data Models are designed and developed for a business audience.
- The conceptual model is developed independently of hardware specifications like data storage capacity, location or software specifications like DBMS vendor and technology. The focus is to represent data as a user will see it in the “real world.”
- Conceptual data models known as Domain models create a common vocabulary for all stakeholders by establishing basic concepts and scope

2**. Representational Data Model**

This type of data model is used to represent only the logical part of the database and does not represent the physical structure of the database. The representational data model allows us to focus primarily, on the design part of the database. A popular representational model is a Relational model. The relational Model consists of Relational Algebra and Relational Calculus. In the Relational Model, we basically use tables to represent our data and the relationships between them. It is a theoretical concept whose practical implementation is done in Physical Data Model.

The advantage of using a Representational data model is to provide a foundation to form the base for the Physical model.

Characteristics of Representational Data Model

- Represents the logical structure of the database.
- Relational models like Relational Algebra and Relational Calculus are commonly used.
- Uses tables to represent data and relationships.
- Provides a foundation for building the physical data model.

3**. Physical Data Model**

The physical Data Model is used to practically implement Relational Data Model. Ultimately, all data in a database is stored physically on a secondary storage device such as discs and tapes. This is stored in the form of files, records, and certain other data structures. It has all the information on the format in which the files are present and the structure of the databases, the presence of external data structures, and their relation to each other. Here, we basically save tables in memory so they can be accessed efficiently. In order to come up with a good physical model, we have to work on the relational model in a better way. Structured Query Language (SQL) is used to practically implement Relational Algebra.

This Data Model describes HOW the system will be implemented using a specific DBMS system. This model is typically created by DBA and developers. The purpose is actual implementation of the database.

Characteristics of a physical data model:

- The physical data model describes data need for a single project or application though it maybe integrated with other physical data models based on project scope.
- Data Model contains relationships between tables that which addresses cardinality and nullability of the relationships.
- Developed for a specific version of a DBMS, location, data storage or technology to be used in the project.
- Columns should have exact datatypes, lengths assigned and default values.
- Primary and Foreign keys, views, indexes, access profiles, and authorizations, etc. are defined

Some Other Data Models

1. Hierarchical Model

The hierarchical Model is one of the oldest models in the data model which was developed by IBM, in the 1950s. In a hierarchical model, data are viewed as a collection of tables, or we can say segments that form a hierarchical relation. In this, the data is organized into a tree-like structure where each record consists of one parent record and many children. Even if the segments are connected as a chain-like structure by logical associations, then the instant structure can be a fan structure with multiple branches. We call the illogical associations as directional associations.

2. Network Model

The Network Model was formalized by the Database Task group in the 1960s. This model is the generalization of the hierarchical model. This model can consist of multiple parent segments and these segments are grouped as levels but there exists

a logical association between the segments belonging to any level. Mostly, there exists a many-to-many logical association between any of the two segments.

3. Object-Oriented Data Model

In the Object-Oriented Data Model, data and their relationships are contained in a single structure which is referred to as an object in this data model. In this, real-world problems are represented as objects with different attributes. All objects have multiple relationships between them. Basically, it is a combination of Object Oriented programming and a Relational Database Model.

4. Semi-Structured Data Model

Semi-Structured data models deal with the data in a flexible way. Some entities may have extra attributes and some entities may have some missing attributes. Basically, you can represent data here in a flexible way.

Advantages of Data Models

- Data Models help us in representing data accurately.
- It helps us in finding the missing data and also in minimizing Data Redundancy.
- Data Model provides data security in a better way.
- The data model should be detailed enough to be used for building the physical database.
- The information in the data model can be used for defining the relationship between tables, primary and foreign keys, and stored procedures.

Disadvantages of Data Models

- In the case of a vast database, sometimes it becomes difficult to understand the data model.
- You must have the proper knowledge of SQL to use physical models.
- Even smaller change made in structure require modification in the entire application.
- There is no set data manipulation language in DBMS.
- To develop Data model one should know physical data stored characteristics.

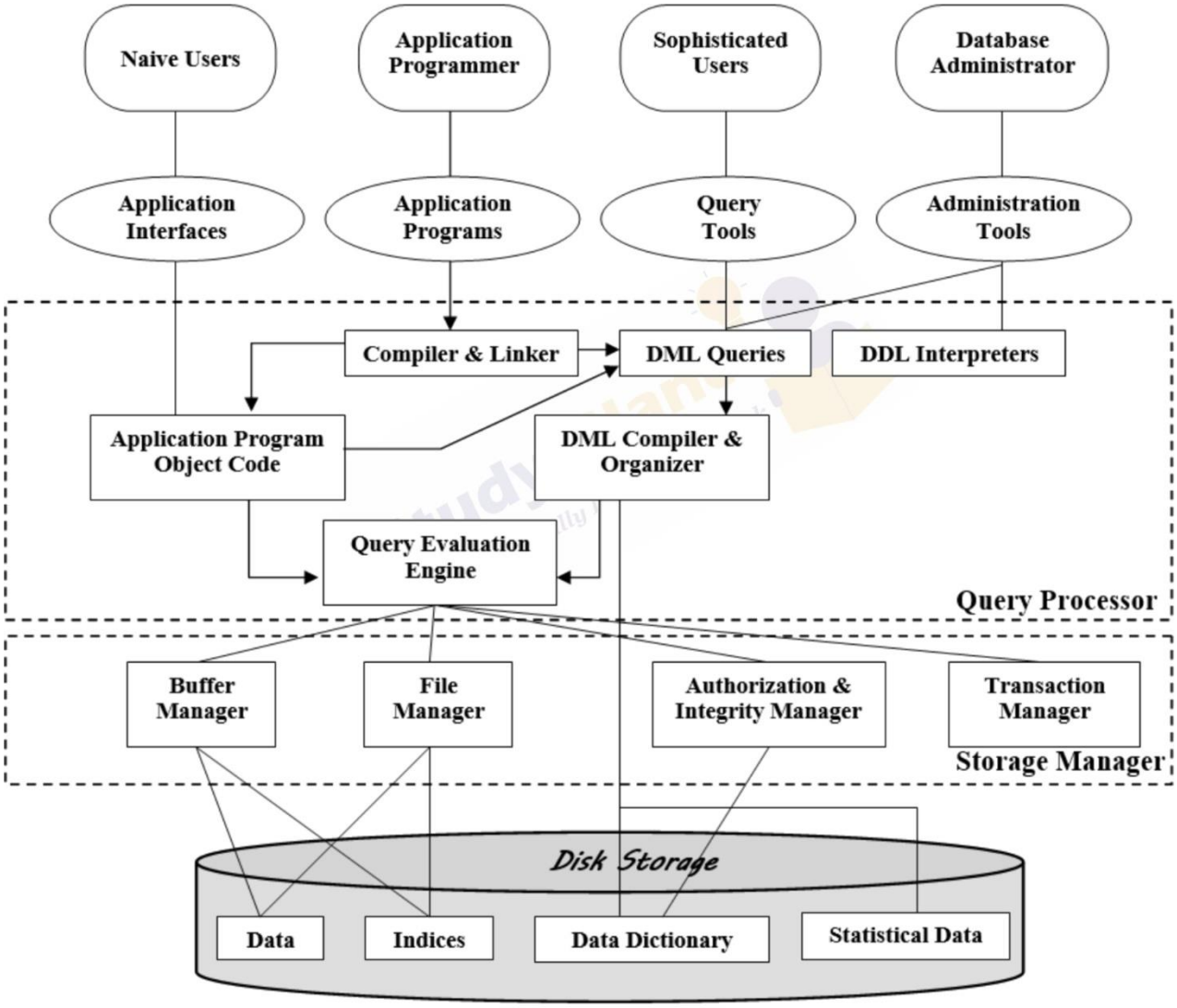
Database System Architecture

The typical structure of DBMS is based on Relational data model.

The top part of the architecture shows application interfaces used by naive users, application programs created by application programmers, query tools used by sophisticated users and administration tools used by database administrator.

The lowest part of the architecture is for disk storage.

The Middle two parts (**Query processor and storage manager**) are important components of database architecture.



Query processor:

The interactive query processor helps the database system to simplify and facilitate access to data. It consists of DDL(**Data Definition Language**) interpreter, DML(**Data Manipulation Language**) compiler and query evaluation engine.

The following are various functionalities and components of query processor

- **DDL interpreter**: This is basically a translator which interprets the DDL statements in data dictionaries.
- **DML compiler**: It translates DML statements query language into an evaluation plan. This plan consists of the instructions which query evaluation engine understands.
- **Query evaluation engine**: It executes the low-level instructions generated by the DML compiler.

When a user issues a query, the parsed query is presented to a query optimizer, which uses information about how the data is stored to produce an efficient execution plan for evaluating the query. An execution plan is a blueprint for evaluating a query. It is evaluated by query evaluation engine.

Storage manager:

Storage manager is the component of database system that provides interface between the low level data stored in the database and the application programs and queries submitted to the system.

The storage manager is responsible for storing, retrieving, and updating data in the database. The storage manager components include

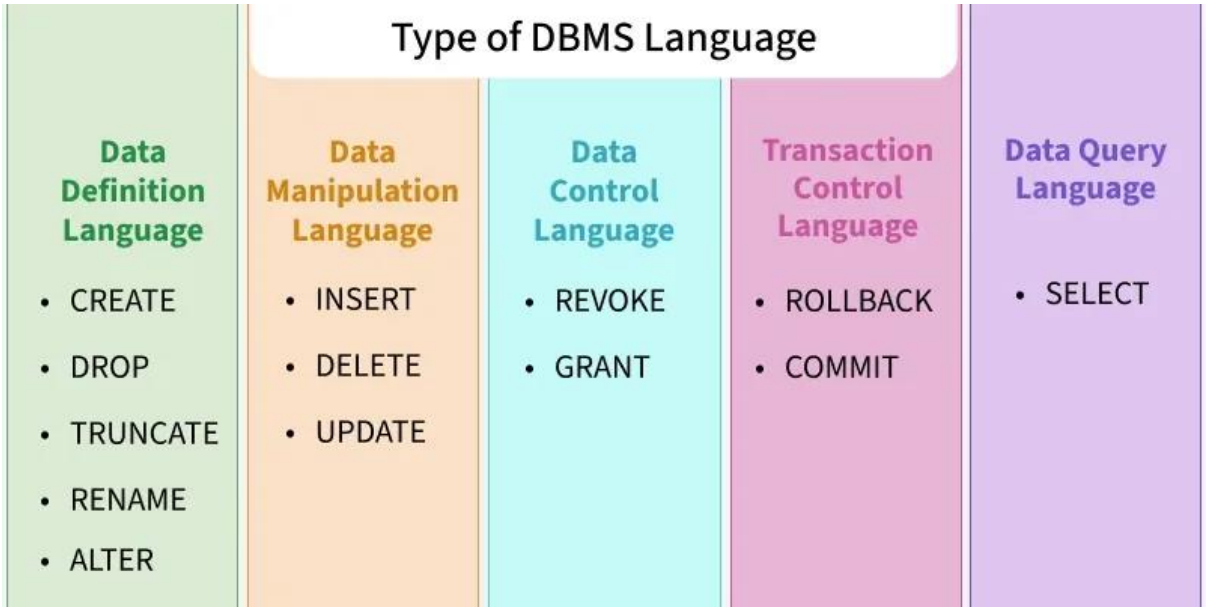
- **Authorization and integrity manager**: Validates the users who want to access the data and tests for integrity constraints.
- **Transaction manager**: Ensures that the database remains in consistent despite of system failures and concurrent transaction execution proceeds without conflicting.
- **File manager**: Manages allocation of space on disk storage and representation of the information on disk.
- **Buffer manager**: Manages the fetching of data from disk storage into main memory. The buffer manager also decides what data to cache in main memory. Buffer manager is a crucial part of database system.

Storage manager implements several data structures such as

- **Data files**: Used for storing database itself.
- **Data dictionary**: Used for storing metadata, particularly schema of database.
- **Indices**: Indices are used to provide fast access to data items present in the database

Database Languages

Database languages are specialized sets of commands and instructions used to define, manipulate and control data within a database. Each language type plays a distinct role in database management, ensuring efficient storage, retrieval and security of data. The primary database languages include:



1. Data Definition Language (DDL)

DDL is the short name for Data Definition Language, which deals with database schemas and descriptions, of how the data should reside in the database.

- **CREATE:** to create a database and its objects like (table, index, views, store procedure, function and triggers)
- **ALTER:** alters the structure of the existing database
- **DROP:** delete objects from the database
- **TRUNCATE:** remove all records from a table, including all spaces allocated for the records are removed
- **COMMENT:** add comments to the data dictionary
- **RENAME:** rename an object

2. Data Manipulation Language (DML)

DML focuses on manipulating the data stored in the database, enabling users to retrieve, add, update and delete data.

- **INSERT:** insert data into a table
- **UPDATE:** updates existing data within a table
- **DELETE:** Delete all records from a database table
- **MERGE:** UPSERT operation (insert or update)
- **CALL:** call a PL/SQL or Java subprogram
- **EXPLAIN PLAN:** interpretation of the data access path
- **LOCK TABLE:** concurrency Control

3. Data Control Language (DCL)

DCL commands manage access permissions, ensuring data security by controlling who can perform certain actions on the database.

- **GRANT**: Provides specific privileges to a user (e.g., SELECT, INSERT).
- **REVOKE**: Removes previously granted permissions from a user.

4. Transaction Control Language (TCL)
TCL commands oversee transactional data to maintain consistency, reliability and atomicity.
- **ROLLBACK**: Undoes changes made during a transaction.
- **COMMIT**: Saves all changes made during a transaction.
- **SAVEPOINT**: Sets a point within a transaction to which one can later roll back.

5. Data Query Language (DQL)
DQL is a subset of SQL used to retrieve data from a database without modifying it. Its main command is SELECT, which allows users to fetch specific information based on their requirements.

Keys

Keys are fundamental elements of the relational database model that ensure uniqueness, data integrity, and efficient data access.

- They uniquely identify each row in a table.
- They prevent data duplication and maintain consistency.
- They create relationships between different tables.

Importance of Keys in DBMS
Keys are important in a Database Management System (DBMS) for several reasons:

- **Uniqueness:** Keys ensure that each record in a table is unique and can be identified distinctly.
- **Data Integrity:** Keys prevent data duplication and maintain the consistency of the data.
- **Efficient Data Retrieval:** Keys help in creating relationships between tables, allowing faster queries and better data organization.

Without keys, managing large databases would become difficult, and data retrieval would be slow and error-prone.

Types of Database Keys

1. Super Key

The set of one or more attributes (columns) that can uniquely identify a tuple (record) is known as Super Key. It may include extra attributes that aren't important for uniqueness but still uniquely identify the row. For Example, STUD_NO, (STUD_NO, STUD_NAME), etc.
- A super key is a group of single or multiple keys that uniquely identifies rows in a table. It supports NULL values in rows.
- A super key can contain extra attributes that aren’t necessary for uniqueness.

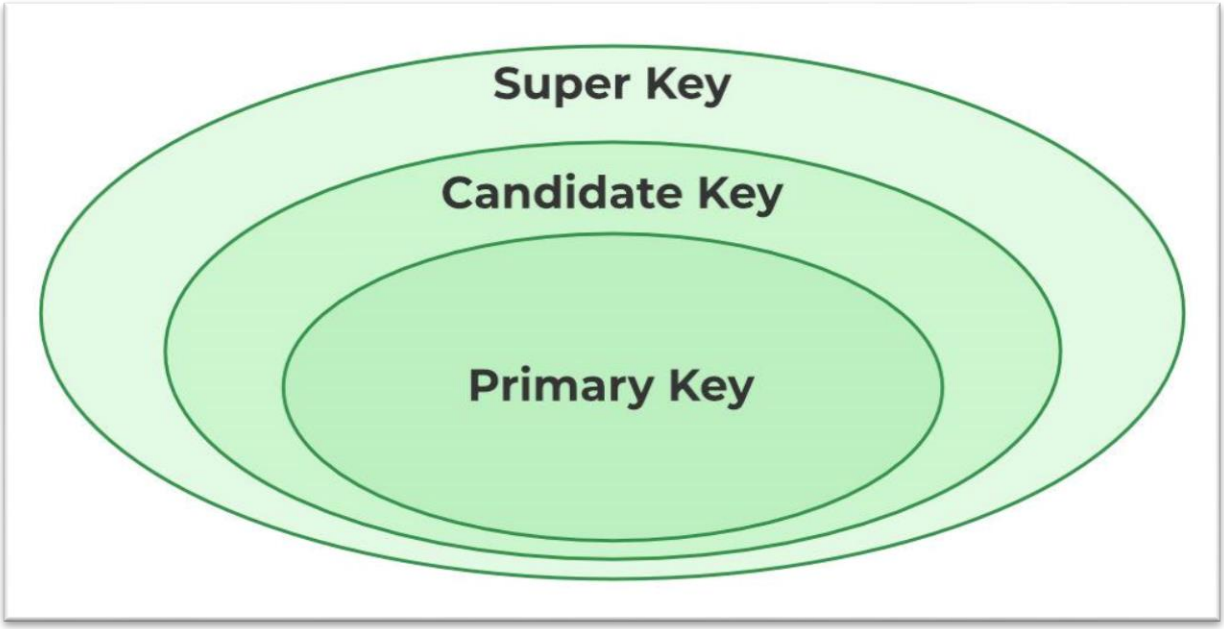
- For example, if the "STUD_NO" column can uniquely identify a student, adding "SNAME" to it will still form a valid super key, though it's unnecessary.

Example: Consider the STUDENT table

STUD_NO	SNAME	ADDRESS	PHONE
1	Tom	New York	123456789
2	John	Los Angeles	223365796
3	Alice	Chicago	175468965

STUDENT Table

A super key could be a combination of STUD_NO and PHONE, as this combination uniquely identifies a student.



Relation between Primary Key, Candidate Key, and Super Key

2. Candidate Key

The minimal set of attributes that can uniquely identify a tuple is known as a candidate key. For Example, STUD_NO in STUDENT relation.

- A candidate key is a minimal super key, meaning it can uniquely identify a record but contains no extra attributes.
- It is a super key with no repeated data is called a candidate key.
- The minimal set of attributes that can uniquely identify a record.
- A candidate key must contain unique values, ensuring that no two rows have the same value in the candidate key's columns.
- Every table must have at least a single candidate key.
- A table can have multiple candidate keys but only one primary key.

Example: For the STUDENT table below, STUD_NO can be a candidate key, as it uniquely identifies each record.

STUD_NO	SNAME	ADDRESS	PHONE
1	Tom	New York	123456789
2	John	Los Angeles	223365796
3	Alice	Chicago	175468965

STUDENT Table

Table: STUDENT_COURSE

STUD_NO	TEACHER_NO	COURSE_NO
1	001	C001
2	056	C005

STUDENT_COURSE Table

A composite candidate key example: {STUD_NO, COURSE_NO} can be a candidate key for a STUDENT_COURSE table.

3. Primary Key

A primary key is chosen from the set of candidate keys to uniquely identify each record in a table. For example, in the STUDENT table, both STUD_NO and STUD_PHONE can be candidate keys, but STUD_NO is selected as the primary key.

- It uniquely identifies every tuple (row) and does not allow duplicate values.
- It cannot be NULL, as each record must have a valid identifier.
- It may be single-column or composite (made of multiple columns).
- Databases often organize data using the primary key to allow faster access and searching.

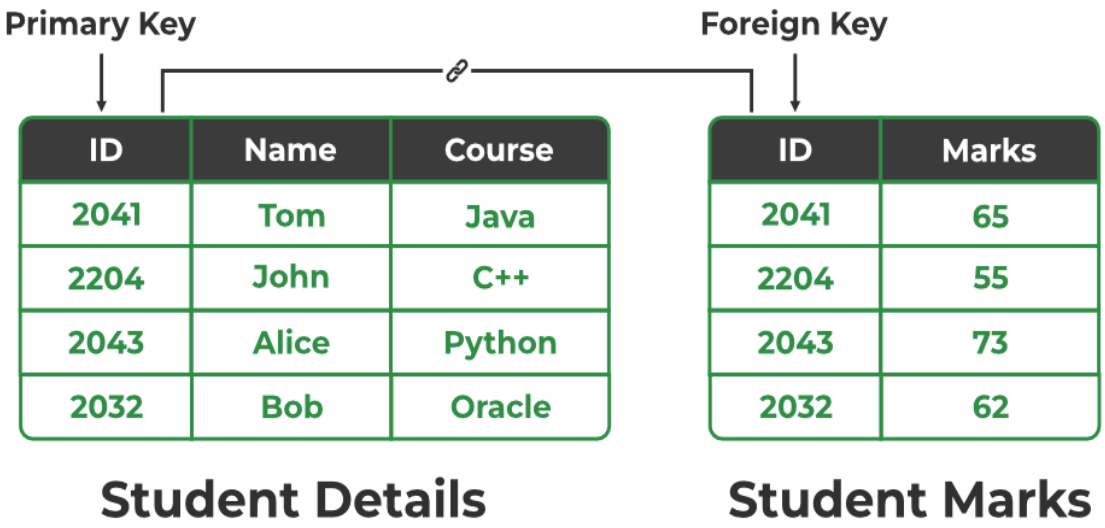
Example: The STUDENT table has the structure Student(STUD_NO, SNAME, ADDRESS, PHONE), where STUD_NO is the primary key.

STUD_NO	SNAME	ADDRESS	PHONE
1	Tom	New York	123456789
2	John	Los Angeles	223365796
3	Alice	Chicago	175468965

STUDENT Table

4. Foreign Key

A foreign key is an attribute in one table that refers to the primary key in another table. The table that contains the foreign key is called the referencing table and the table that is referenced is called the referenced table.



Relation between Primary Key and Foreign Key

- A foreign key in one table points to the primary key in another table, establishing a relationship between them.
- It helps connect two or more tables, enabling you to create relationships between them. This is important for maintaining data integrity and preventing data redundancy.
- They act as a cross-reference between the tables.

Example: Consider the STUDENT_COURSE table

STUD_NO	TEACHER_NO	COURSE_NO
1	001	C001
2	056	C005

STUDENT_COURSE Table

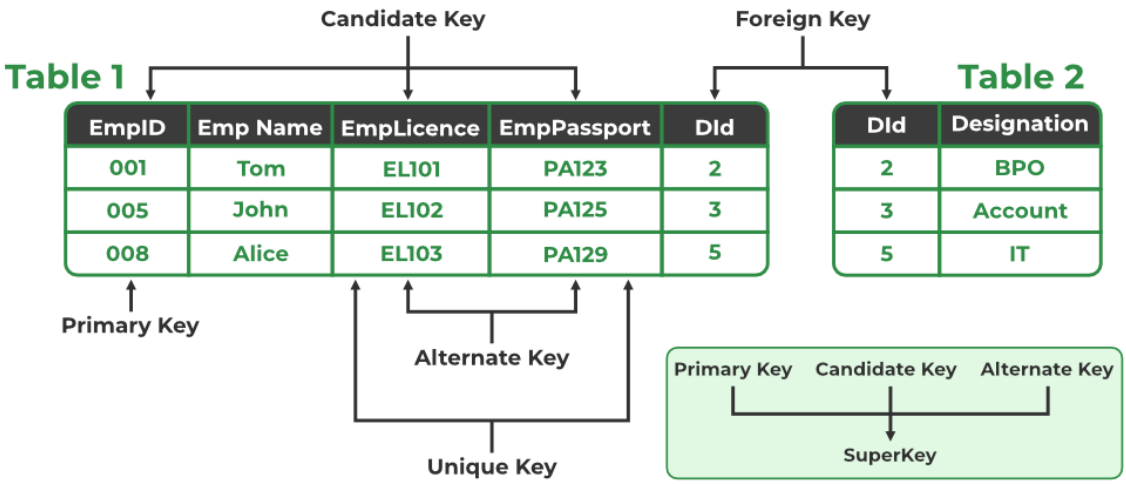
- STUD_NO in the STUDENT_COURSE table is a foreign key that refers to the STUD_NO primary key of the STUDENT table.
- Unlike a primary key, a foreign key can contain duplicate values and may be NULL. For example, STUD_NO appears multiple times in STUDENT_COURSE because a student can enroll in more than one course.
- However, STUD_NO in the STUDENT table is a primary key, so it must always be unique and non-NULL.

5. Composite Key

Sometimes, a single column is not enough to uniquely identify all records in a table, so a combination of multiple attributes is used. An optimal set of such attributes is chosen to ensure that every row is uniquely identifiable.

- It acts as a primary key if there is no primary key in a table
- Two or more attributes are used together to make a composite key .
- Different combinations of attributes may give different accuracy in terms of identifying the rows uniquely.

Example: In the STUDENT_COURSE table, {STUD_NO, COURSE_NO} can form a composite key to uniquely identify each record.



Constraints

Constraints in DBMS (Database Management Systems) encompass rules or conditions applied to database data, ensuring data integrity, consistency, and adherence to business rules.

They define limitations and requirements that data must meet, preventing the entry of invalid or inconsistent data. Constraints serve as pre-established rules governing the behavior and relationships of data in a database, contributing to the maintenance of accuracy and reliability.

The purpose of **constraints** is to enforce data quality and thwart data inconsistencies, thereby bolstering the overall data integrity and reliability of the database. Constraints set boundaries for data values, relationships between entities, uniqueness requirements, and more.

Through constraint enforcement, DBMS ensures data conforms to predefined standards and business rules, fortifying the database's robustness and reliability.

Types of Constraints

Within relational databases, there are primarily five types of constraints, commonly referred to as relational constraints. These include:

1. NOT NULL
Ensures that a column **cannot have NULL values**.
name VARCHAR(50) NOT NULL

2. UNIQUE
Ensures that **all values in a column are different**.
email VARCHAR(50) UNIQUE

3. PRIMARY KEY

- Combination of **NOT NULL + UNIQUE**
- Uniquely identifies each record in a table

student_id INT PRIMARY KEY

4. FOREIGN KEY

- Creates a **relationship between two tables**
- Ensures **referential integrity**

FOREIGN KEY (dept_id) REFERENCES Department(dept_id)

4. CHECK
Ensures that column values satisfy a **specific condition**.
age INT CHECK (age >= 18)

6. DEFAULT
Assigns a **default value** if no value is provided.
status VARCHAR(10) DEFAULT 'Active'

7. DOMAIN Constraint
Restricts values to a **specific domain** (data type, size, range).
salary DECIMAL(8,2)

Example Table with Constraints

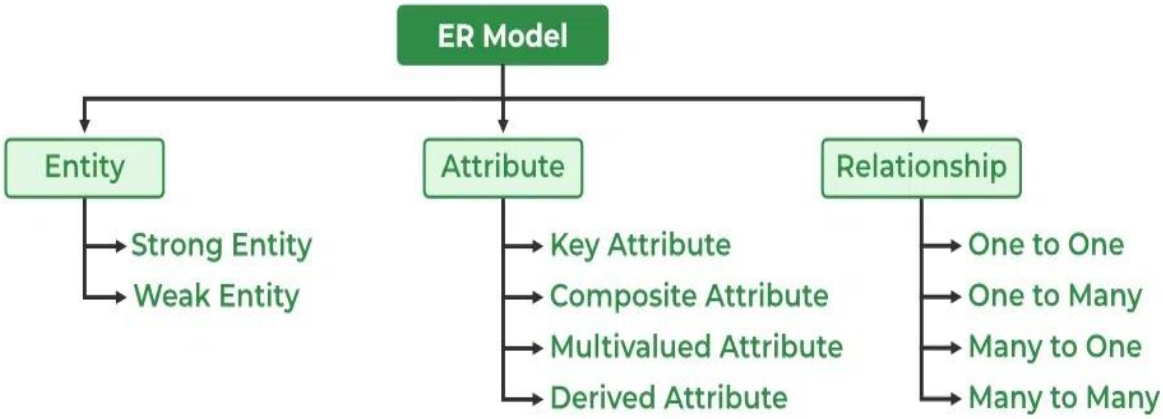
```
CREATE TABLE Student (
    student_id INT PRIMARY KEY,
```

```
    name VARCHAR(50) NOT NULL,
    age INT CHECK (age >= 18),
    email VARCHAR(50) UNIQUE,
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);
```

Database Design and ER Model

Introduction of ER Model

- The Entity-Relationship Model (ER Model) is a conceptual model for designing a database. This model represents the logical structure of a database, including entities, their attributes, and relationships between them.
  Entity: An object that is stored as data. E.g: *Student*, *Course,* or *Company*.
  Attribute: Properties that describe an entity. E.g: *StudentID*, *CourseName*, or *EmployeeEmail*.
  Relationship: A connection between entities. E.g: *Student* enrolls in a *Course*.



Components of ER Diagram

The graphical representation of this model is called an Entity-Relation Diagram (ERD).

What is an Entity Relationship Diagram (ER Diagram)?

An ER diagram shows the relationship among entity sets. An entity set is a group of similar entities and these entities can have attributes. In terms of DBMS, an entity is a table or attribute of a table in database, so by showing relationship among tables

and their attributes, ER diagram shows the complete logical structure of a database. Lets have a look at a simple ER diagram to understand this concept.

Facts about ER Diagram Model:

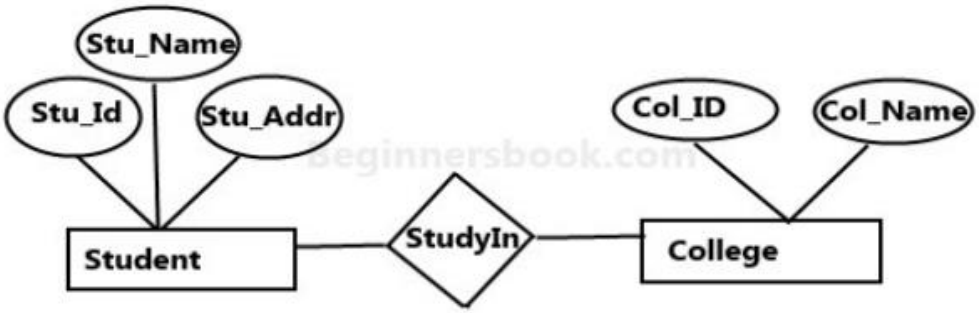
ER model allows you to draw Database Design It is an easy to use graphical tool for modeling data Widely used in Database Design It is a GUI representation of the logical structure of a Database It helps you to identifies the entities which exist in a system and the relationships between those entities

Why use ER Diagrams?

Here, are prime reasons for using the ER Diagram Helps you to define terms related to entity relationship modeling Provide a preview of how all your tables should connect, what fields are going to be on each table Helps to describe entities, attributes, relationships ER diagrams are translatable into relational tables which allows you to build databases quickly ER diagrams can be used by database designers as a blueprint for implementing data in specific software applications

A simple ER Diagram:

In the following diagram we have two entities Student and College and their relationship. The relationship between Student and College is many to one as a college can have many students however a student cannot study in multiple colleges at the same time. Student entity has attributes such as Stu_Id, Stu_Name & Stu_Addr and College entity has attributes such as Col_ID & Col_Name.

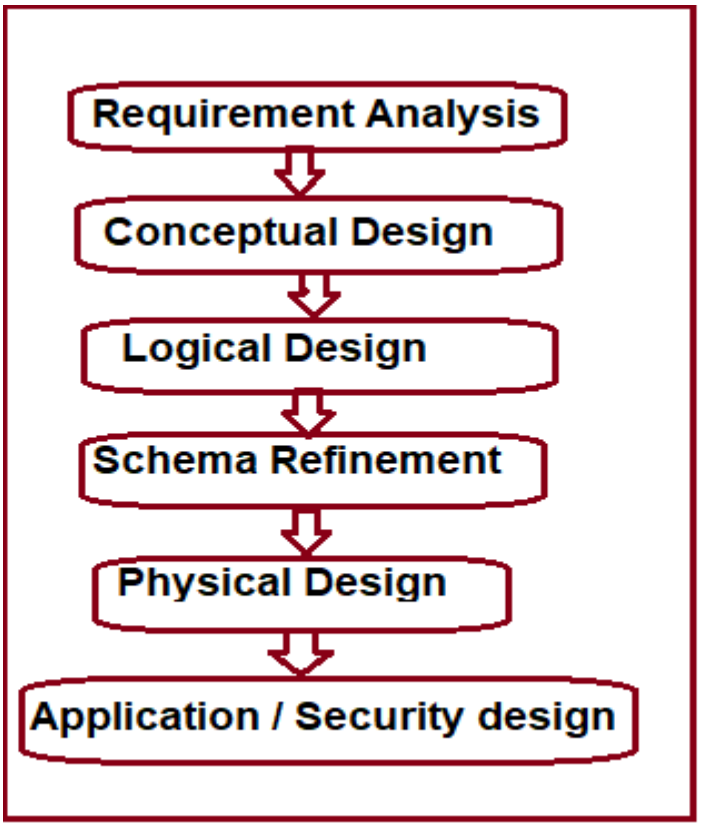


Sample E-R Diagram

Here are the geometric shapes and their meaning in an E-R Diagram. We will discuss these terms in detail in the next section (Components of a ER Diagram) of this guide so don’t worry too much about these terms now, just go through them once.

Rectangle: Represents Entity sets.
Ellipses: Attributes
Diamonds: Relationship
Set Lines: They link attributes to Entity Sets and Entity sets to Relationship Set
Double Ellipses: Multivalued Attributes
Dashed Ellipses: Derived Attributes
Double Rectangles: Weak Entity Sets
Double Lines: Total participation of an entity in a relationship set

ER Model in Database Design Process



We typically follow the below steps for designing a database for an application.

1. Requirement Analysis

- Collect requirements from users, clients, and stakeholders
- Identify **data needs, operations, constraints, and security requirements**

2. Conceptual Design

- Create a **high-level data model**
- Represent data using **ER diagrams**

3. Logical Design

- Convert conceptual design into a **logical schema**
- Map ER diagram into **relational tables**

4. Schema Refinement (Normalization)

- Improve database structure
- Remove redundancy and anomalies

5. Physical Design

- Decide **how data is physically stored**
- Optimize performance

6. Application / Security Design

- Design user interfaces and applications
- Implement **security and authorization**

Use ER Diagrams In DBMS

- ER diagrams represent the E-R model in a database, making them easy to convert into relations (tables).
- These diagrams serve the purpose of real-world modeling of objects which makes them intently useful.
- Unlike technical schemas, ER diagrams require no technical knowledge of the underlying DBMS used.
- They visually model data and its relationships, making complex systems easier to understand.

Symbols Used in ER Model

ER Model is used to model the logical view of the system from a data perspective which consists of these symbols:

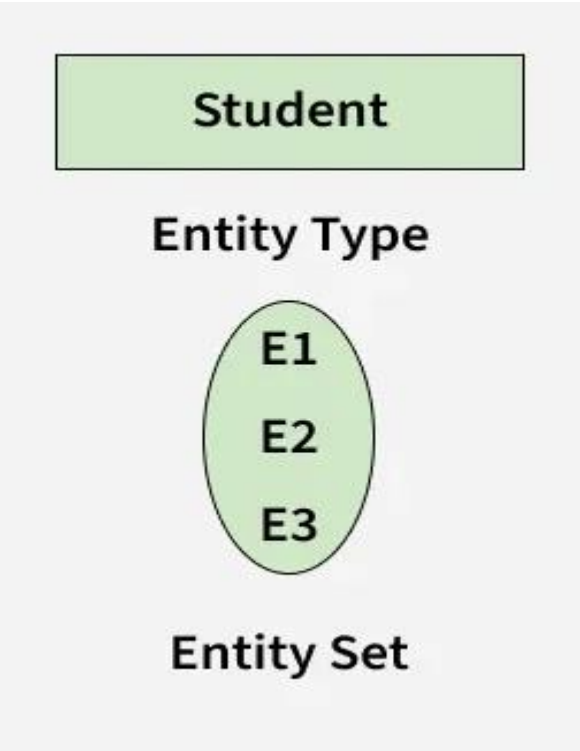
Figures	Symbols	Represents
Rectangle		Entities in ER Model
Ellipse		Attributes in ER Model
Diamond		Relationships among Entities
Line		Attributes to Entities and Entity Sets with Other Relationship Types
Double Ellipse		Multi-Valued Attributes
Double Rectangle		Weak Entity

Symbols used in ER Diagram

Entity
An Entity represents a real-world object, concept or thing about which data is stored in a database. It act as a building block of a database. Tables in relational database represent these entities.
Example of entities:
Real-World Objects: Person, Car, Employee etc.
Concepts: Course, Event, Reservation etc.
Things: Product, Document, Device etc.

The entity type defines the structure of an entity, while individual instances of that type represent specific entities.

Entity Set
An entity refers to an individual object of an entity type, and the collection of all entities of a particular type is called an entity set. For example, E1 is an entity that belongs to the entity type "Student," and the group of all students forms the entity set.
In the ER diagram below, the entity type is represented as:



Entity Set

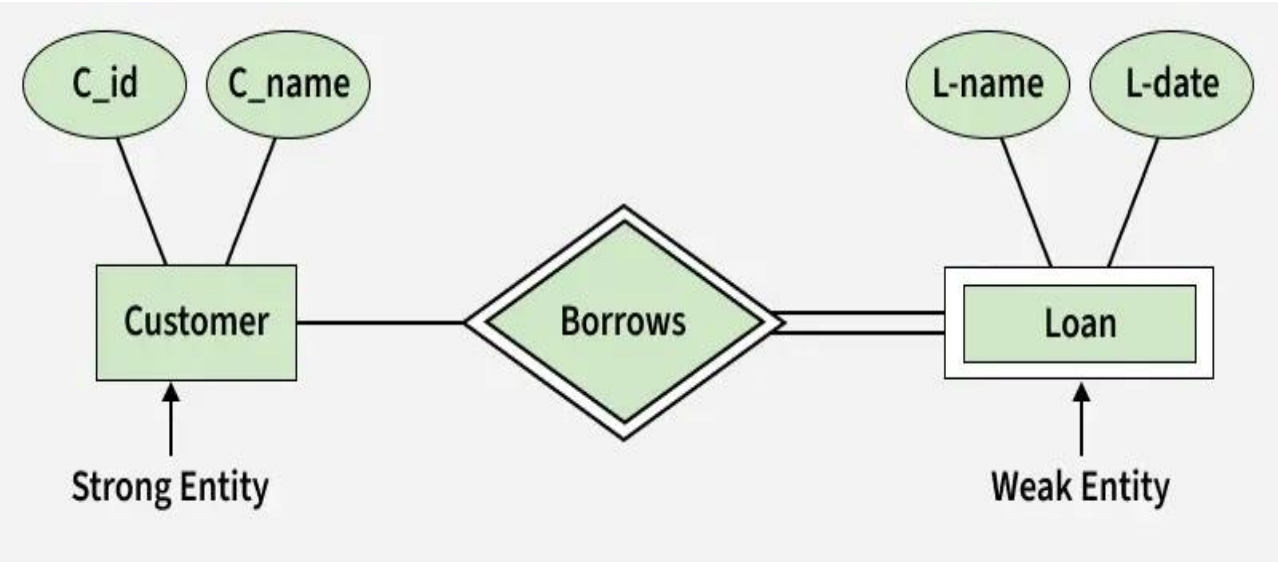
We can represent the entity sets in an ER Diagram but we can't represent individual entities because an entity is like a row in a table, and an ER diagram shows the structure and relationships of data, not specific data entries (like rows and columns). An ER diagram is a visual representation of the data model, not the actual data itself.

Types of Entity
There are two main types of entities:

1. Strong Entity
A Strong Entity is a type of entity that has a key Attribute that can uniquely identify each instance of the entity. A Strong Entity does not depend on any other Entity in the Schema for its identification. It has a primary key that ensures its uniqueness and is represented by a rectangle in an ER diagram.

2. Weak Entity
A Weak Entity cannot be uniquely identified by its own attributes alone. It depends on a strong entity to be identified. A weak entity is associated with an identifying entity (strong entity), which helps in its identification. A weak entity are represented by a double rectangle. The participation of weak entity types is always total. The relationship between the weak entity type and its identifying strong entity type is called identifying relationship and it is represented by a double diamond.

Example:
A company may store the information of dependents (Parents, Children, Spouse) of an Employee. But the dependents can't exist without the employee. So dependent will be a Weak Entity Type and Employee will be identifying entity type for dependent, which means it is Strong Entity Type.



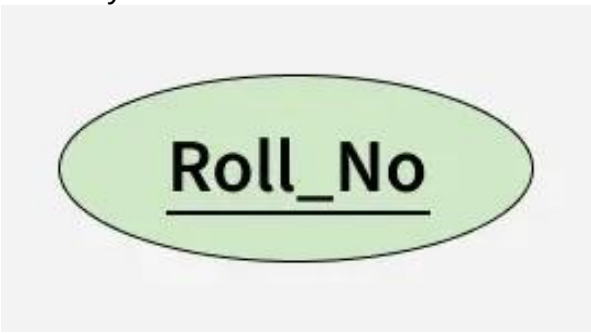
Strong Entity and Weak Entity

Strong Entity Set	**Weak Entity Set**
Strong entity set always has a primary key.	It does not have enough attributes to build a primary key.
It is represented by a rectangle symbol.	It is represented by a double rectangle symbol.
It contains a Primary key represented by the underline symbol.	It contains a Partial Key which is represented by a dashed underline symbol.
The member of a strong entity set is called as dominant entity set.	The member of a weak entity set called as a subordinate entity set.
Primary Key is one of its attributes which helps to identify its member.	In a weak entity set, it is a combination of primary key and partial key of the strong entity set.
In the ER diagram the relationship between two strong entity set shown by using a diamond symbol.	The relationship between one strong and a weak entity set shown by using the double diamond symbol.
The connecting line of the strong entity set with the relationship is single.	The line connecting the weak entity set for identifying relationship is double.

Attributes in ER Model
Attributes are the properties that define the entity type. For example, for a Student entity Roll_No, Name, DOB, Age, Address, and Mobile_No are the attributes that define entity type Student. In ER diagram, the attribute is represented by an oval.

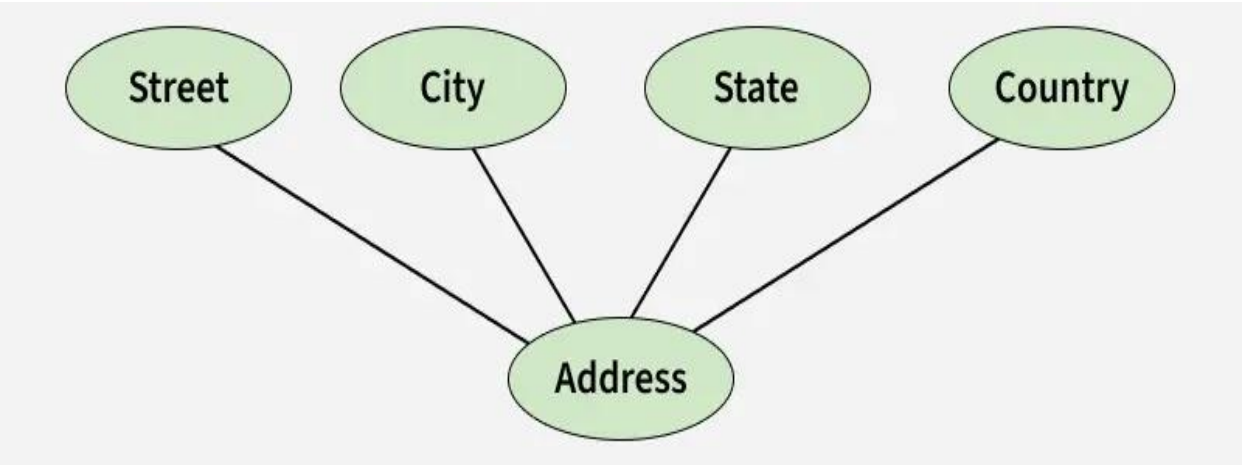
Attribute

Types of Attributes
1. Key Attribute
The attribute which uniquely identifies each entity in the entity set is called the key attribute. For example, Roll_No will be unique for each student. In ER diagram, the key attribute is represented by an oval with an underline.



Key Attribute

2. Composite Attribute
An attribute composed of many other attributes is called a composite attribute. For example, the Address attribute of the student Entity type consists of Street, City, State, and Country. In ER diagram, the composite attribute is represented by an oval comprising of ovals.



Composite Attribute

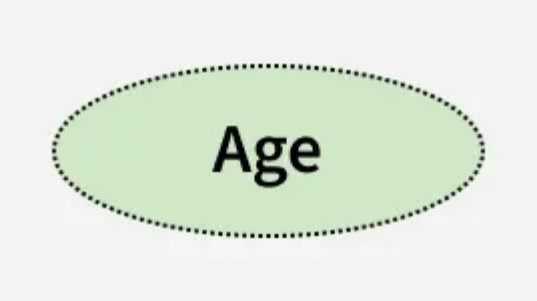
3. Multivalued Attribute

An attribute consisting of more than one value for a given entity. For example, Phone_No (can be more than one for a given student). In ER diagram, a multivalued attribute is represented by a double oval.

Multivalued Attribute

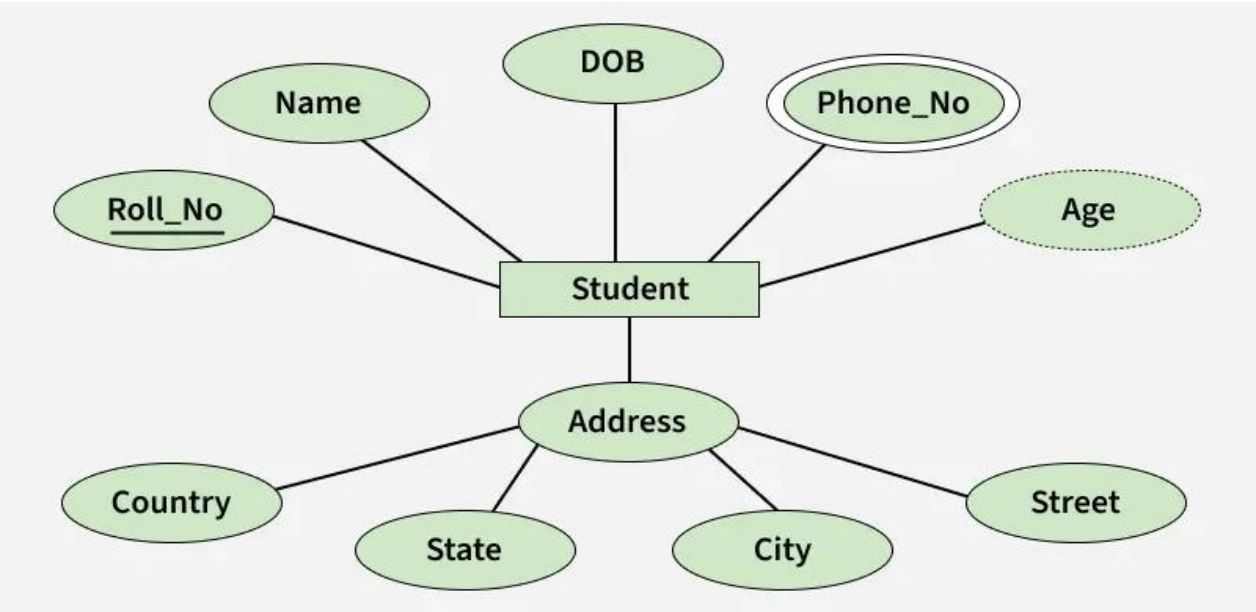
4. Derived Attribute

An attribute that can be derived from other attributes of the entity type is known as a derived attribute. e.g.; Age (can be derived from DOB). In ER diagram, the derived attribute is represented by a dashed oval.



Derived Attribute

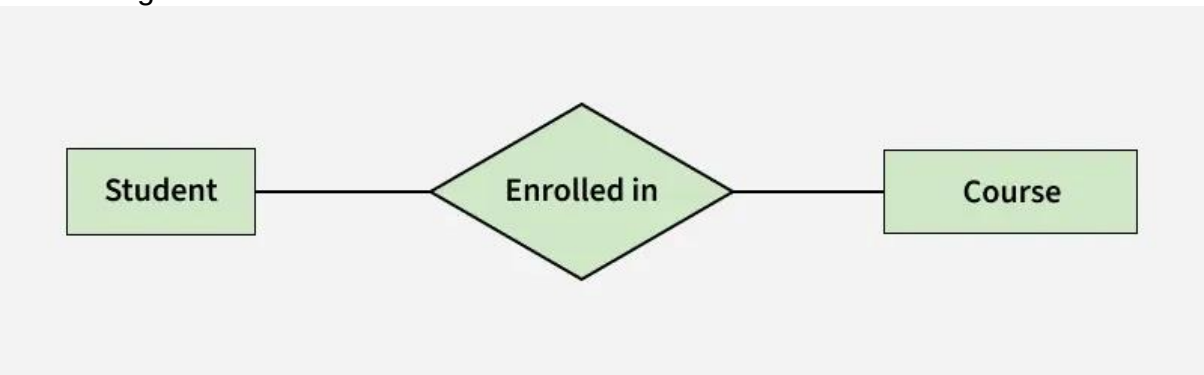
The Complete Entity Type Student with its Attributes can be represented as:



Entity and Attributes

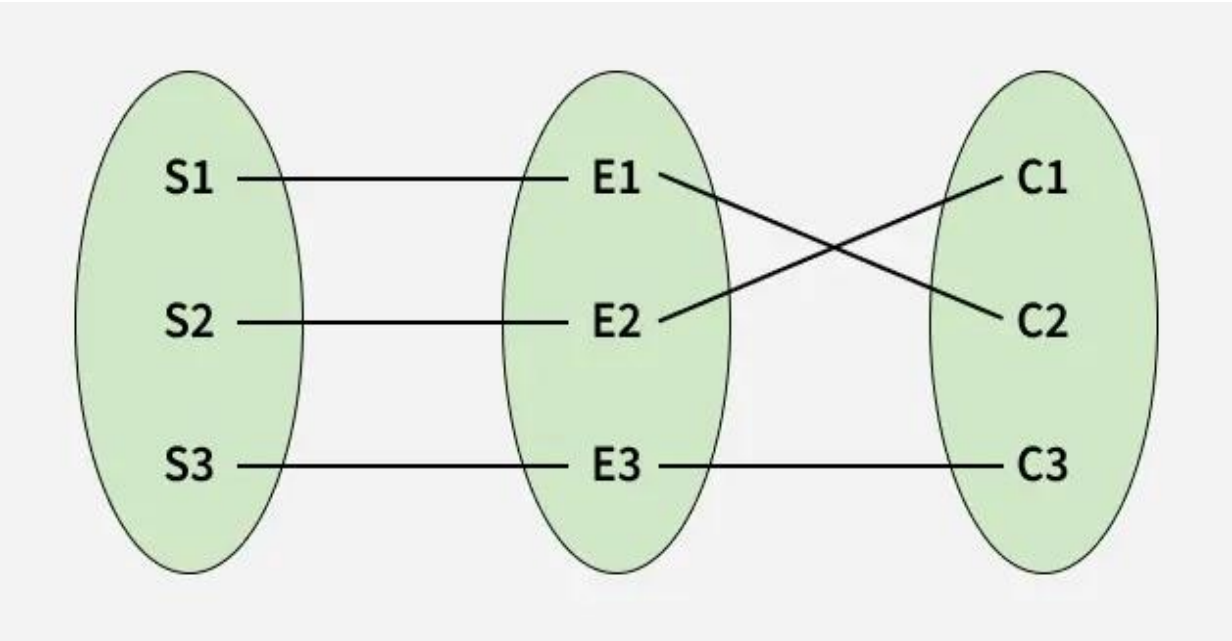
Relationship Type and Relationship Set

A Relationship Type represents the association between entity types. For example, ‘Enrolled in’ is a relationship type that exists between entity type Student and Course. In ER diagram, the relationship type is represented by a diamond and connecting the entities with lines.



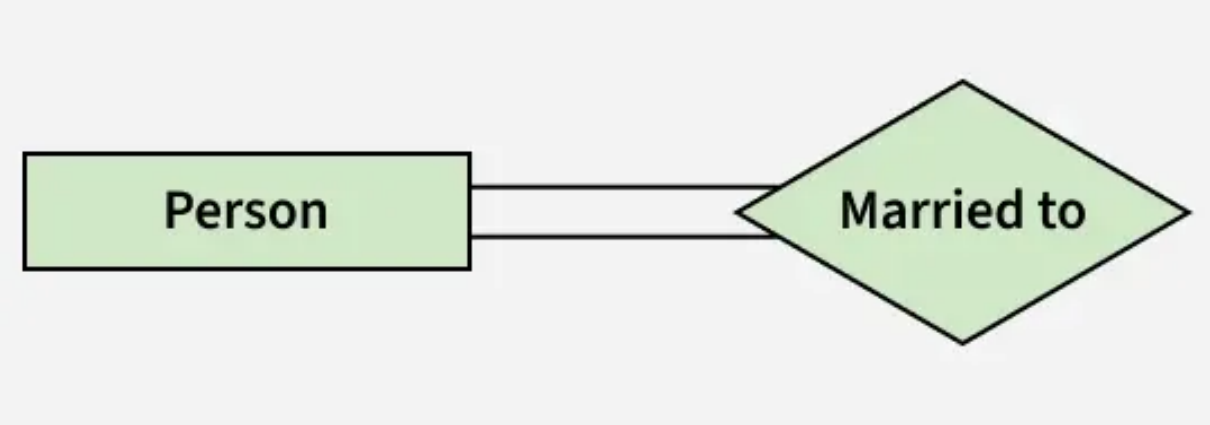
Entity-Relationship Set

A set of relationships of the same type is known as a relationship set. The following relationship set depicts S1 as enrolled in C2, S2 as enrolled in C1, and S3 as registered in C3.



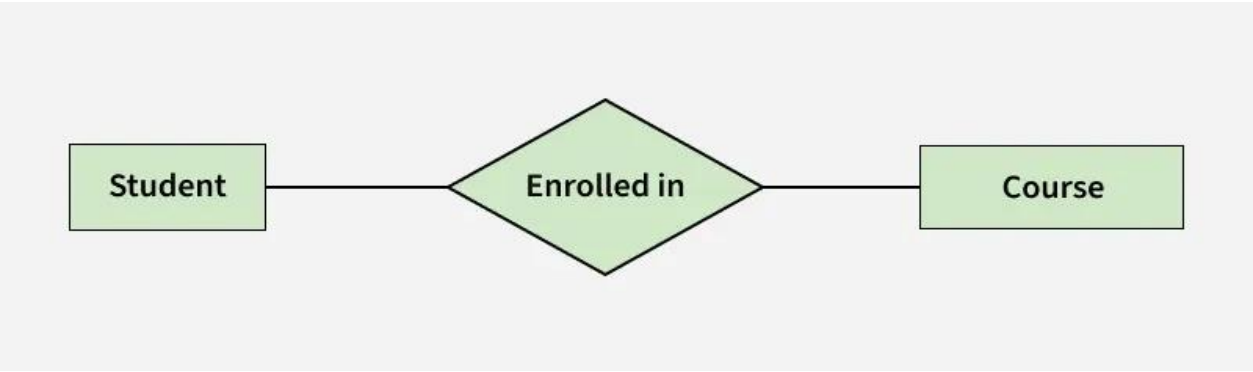
Relationship Set

Degree of a Relationship Set
The number of different entity sets participating in a relationship set is called the degree of a relationship set.
1. Unary/Recursive Relationship: When there is only ONE entity set participating in a relation, the relationship is called a unary relationship. For example, one person is married to only one person.



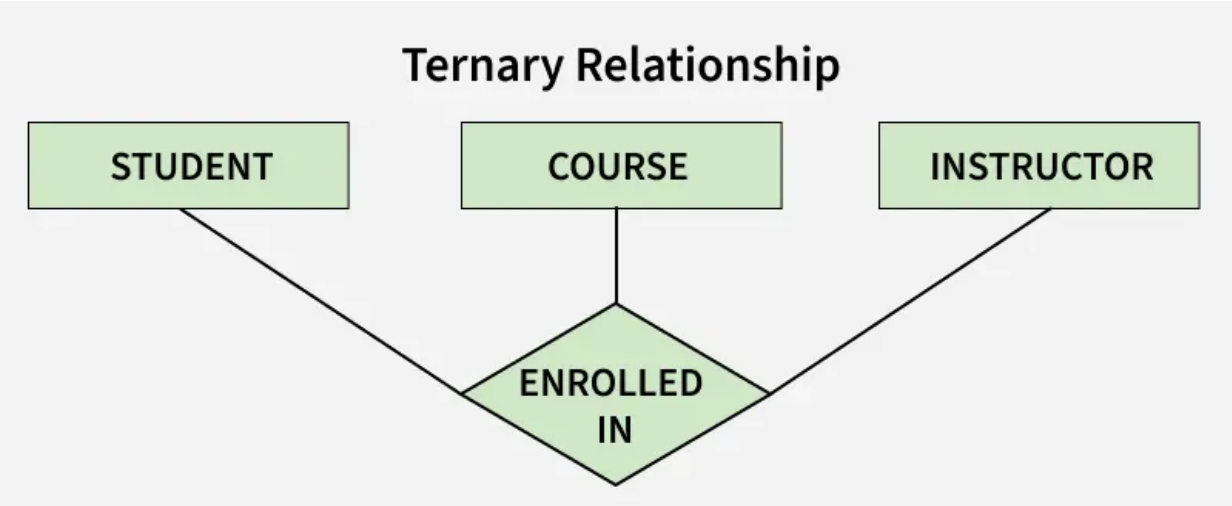
Unary Relationship
Read more about - Recursive Relationship

2. Binary Relationship: When there are TWO entities set participating in a relationship, the relationship is called a binary relationship. For example, a Student is enrolled in a Course.



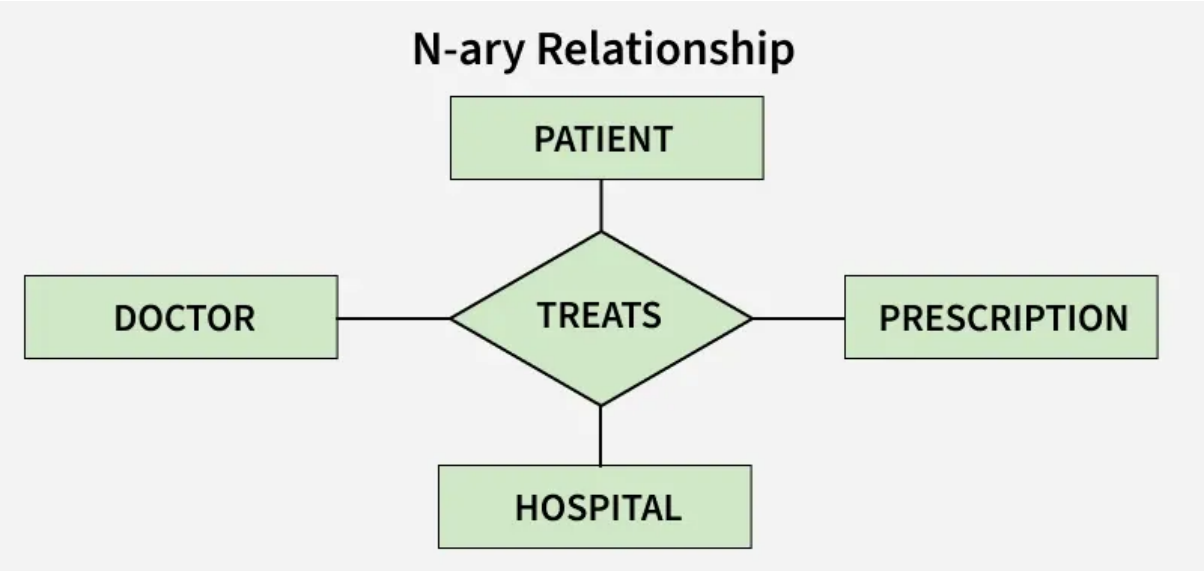
Binary Relationship

3. Ternary Relationship: When there are three entity sets participating in a relationship, the relationship is called a ternary relationship.



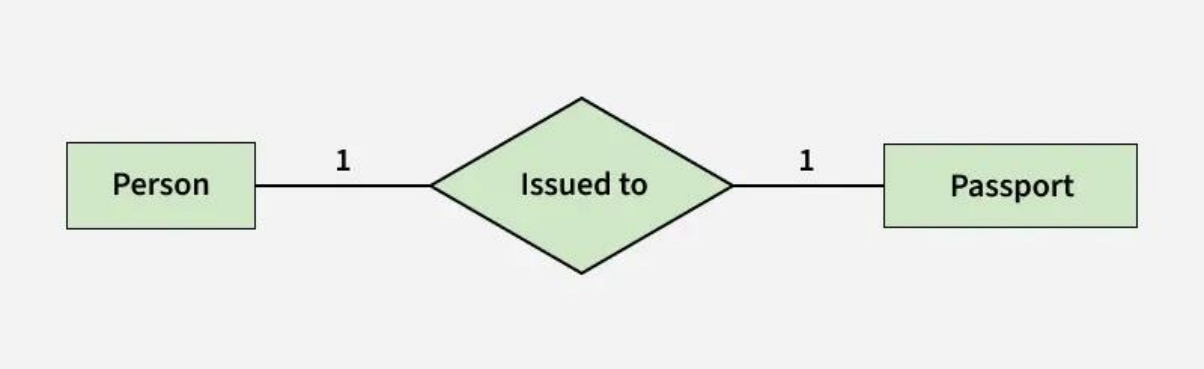
Ternary Relationship

5. **N-ary Relationship:** When there are n entities set participating in a relationship, the relationship is called an n-ary relationship.



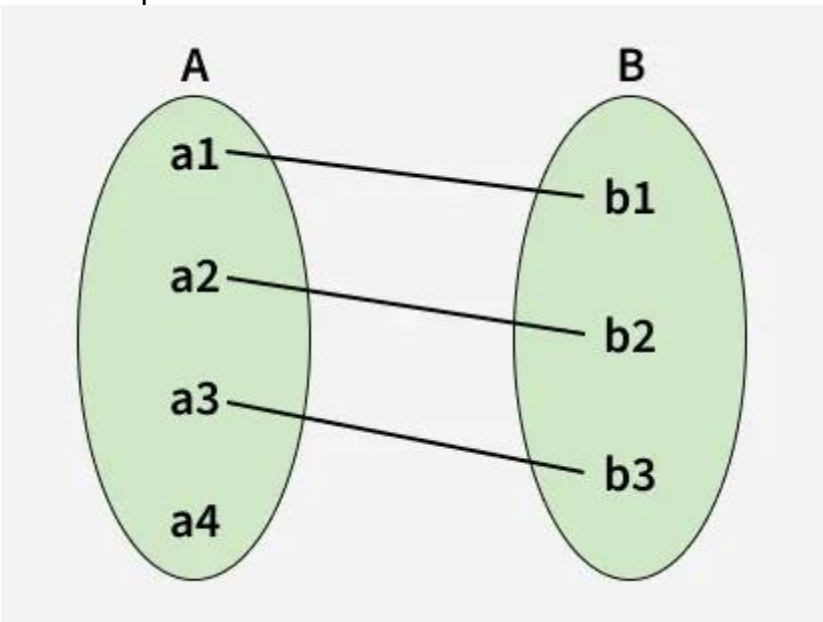
N-ary Relationship

Types of mapping Constraints or relationships in ER Model
The maximum number of times an entity of an entity set participates in a relationship set is known as cardinality.
Cardinality can be of different types:
1. One-to-One
When each entity in each entity set can take part only once in the relationship, the cardinality is one-to-one. Let us assume that one person can be issued only one passport, and one passport is issued to only one person. So, the relationship will be One-to-One (1 : 1), meaning that each person has a single passport, and each passport belongs to a single person.



one to one cardinality

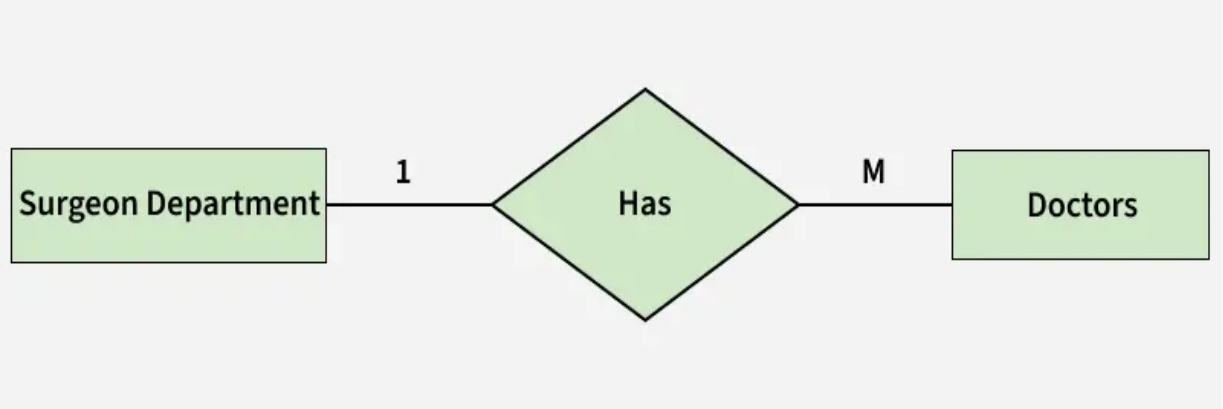
Using Sets, it can be represented as:



Set Representation of One-to-One

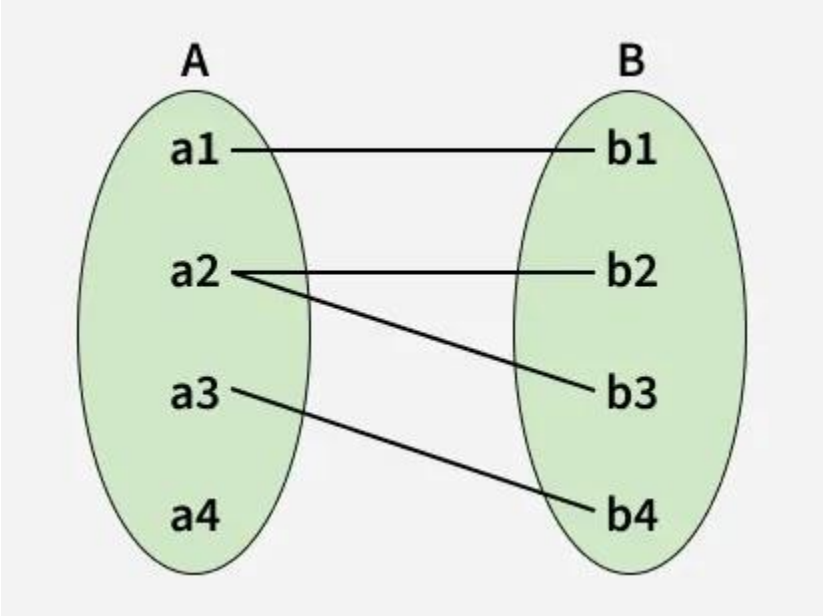
2. One-to-Many
In a one-to-many relationship, one entity can be associated with multiple entities. For example, a single Surgeon Department can have many Doctors. Therefore, the cardinality of this relationship is 1 to M, meaning one department can have many doctors.

one to man



y cardinality

Using sets, one-to-many cardinality can be represented as:

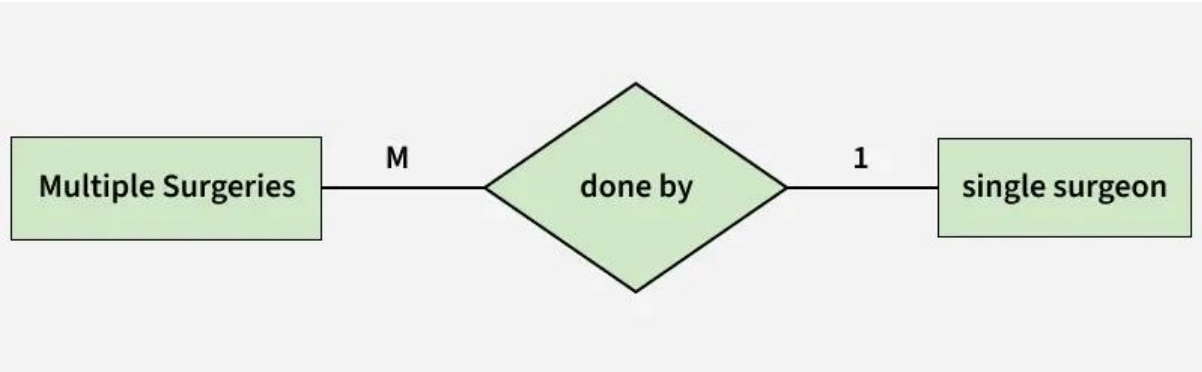


Set Representation of One-to-Many

3. Many-to-One

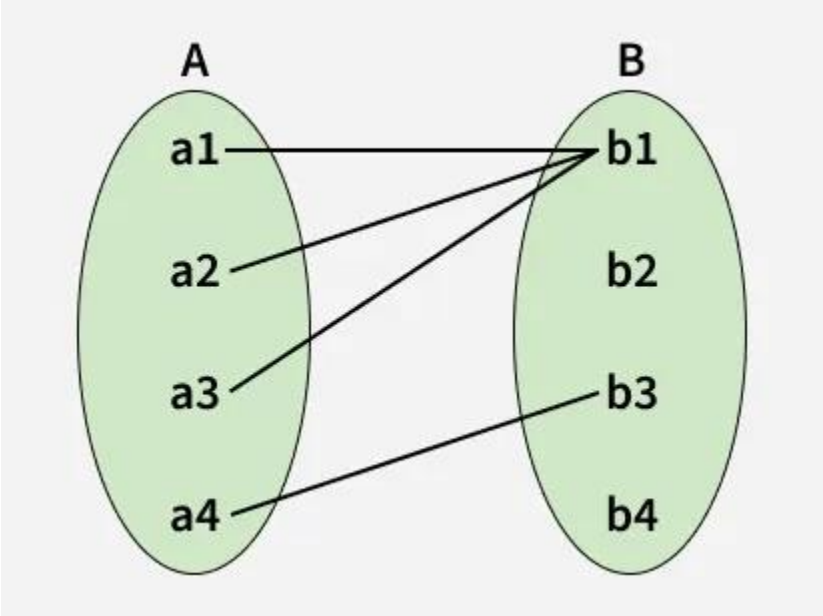
When entities in one entity set can take part only once in the relationship set and entities in other entity sets can take part more than once in the relationship set, cardinality is many to one.

Let us assume that multiple surgeries can be performed by one surgeon, but one surgery is performed by only one surgeon. So, the cardinality will be M to 1, meaning that many surgeries can be done by a single surgeon, but each surgery is done by only one surgeon.



many to one cardinality

Using Sets, it can be represented as:

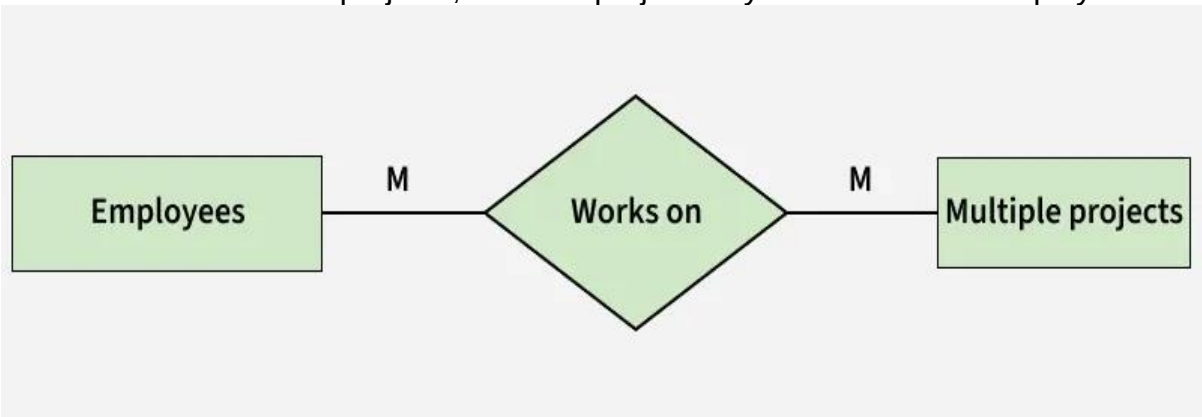


Set Representation of Many-to-One

In this case, each student is taking only 1 course but 1 course has been taken by many students.

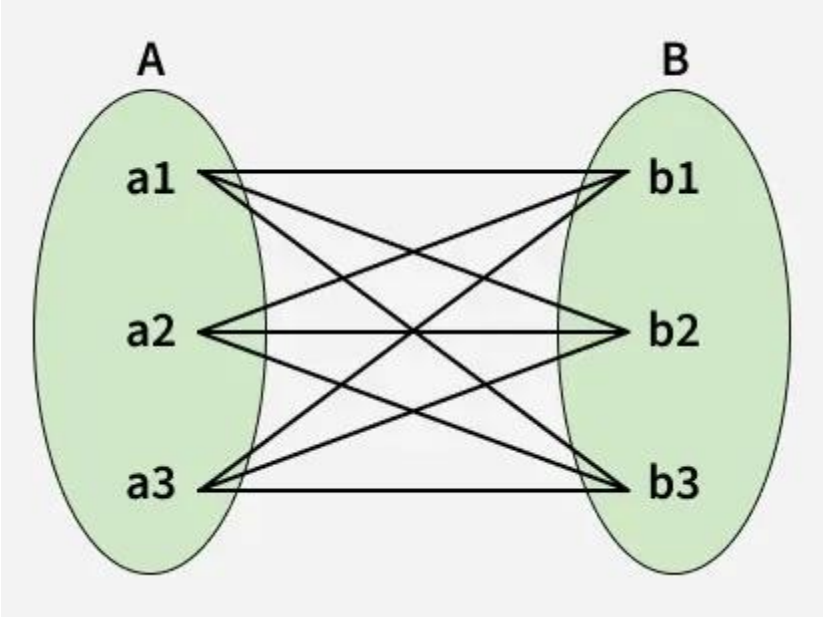
4. Many-to-Many

When entities in all entity sets can take part more than once in the relationship cardinality is many to many. Let us assume that an employee can work on multiple projects and each project can have multiple employees working on it. So, the relationship will be many-to-many (M:N), meaning that one employee may be associated with several projects, and one project may involve several employees.



many to many cardinality

Using Sets, it can be represented as:



Many-to-Many Set Representation

In this example, student S1 is enrolled in C1 and C3 and Course C3 is enrolled by S1, S3, and S4. So it is many-to-many relationships.

How to Draw an ER Diagram:

1. **Identify Entities:** The very first step is to identify all the Entities. Represent these entities in a Rectangle and label them accordingly.
2. **Identify Relationships**: The next step is to identify the relationship between them and represent them accordingly using the Diamond shape. Ensure that relationships are not directly connected to each other.
3. **Add Attributes**: Attach attributes to the entities by using ovals. Each entity can have multiple attributes (such as name, age, etc.), which are connected to the respective entity.
4. **Define Primary Keys:** Assign primary keys to each entity. These are unique identifiers that help distinguish each instance of the entity. Represent them with underlined attributes.
5. **Remove Redundancies**: Review the diagram and eliminate unnecessary or repetitive entities and relationships.
6. **Review for Clarity**: Review the diagram make sure it is clear and effectively conveys the relationships between the entities.

Steps to Create an ERD (E-R Diagram)

Following are the steps to create an ERD.

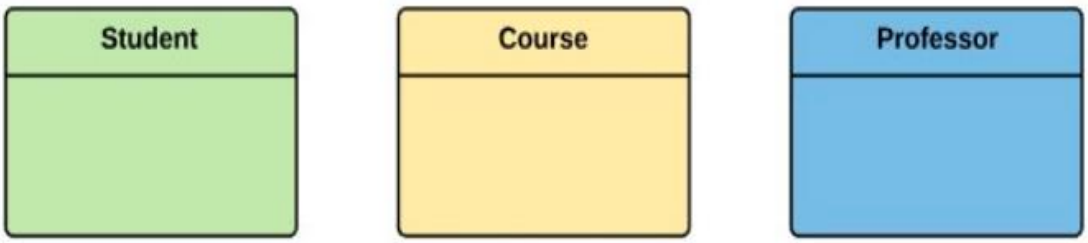
Let's study them with an example:

In a university, a Student enrolls in Courses. A student must be assigned to at least one or more Courses. Each course is taught by a single Professor. To maintain instruction quality, a Professor can deliver only one course

Step 1) Entity Identification

We have three entities

- Student
- Course
- Professor



Step 2) Relationship Identification

We have the following two relationships

- The student is assigned a course
- Professor delivers a course

Step 3) Cardinality Identification

For them problem statement we know that,

- A student can be assigned multiple courses
- A Professor can deliver only one course

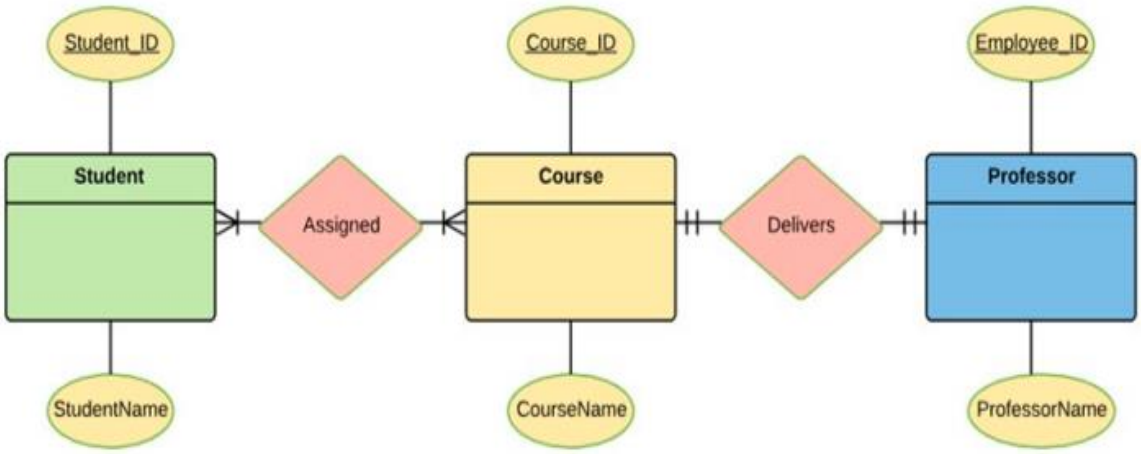
Step 4) Identify Attributes

You need to study the files, forms, reports, data currently maintained by the organization to identify attributes. You can also conduct interviews with various stakeholders to identify entities. Initially, it's important to identify the attributes without mapping them to a particular entity.

Once, you have a list of Attributes, you need to map them to the identified entities. Ensure an attribute is to be paired with exactly one entity. If you think an attribute should belong to more than one entity, use a modifier to make it unique.

Once the mapping is done, identify the primary Keys. If a unique key is not readily available, create one.

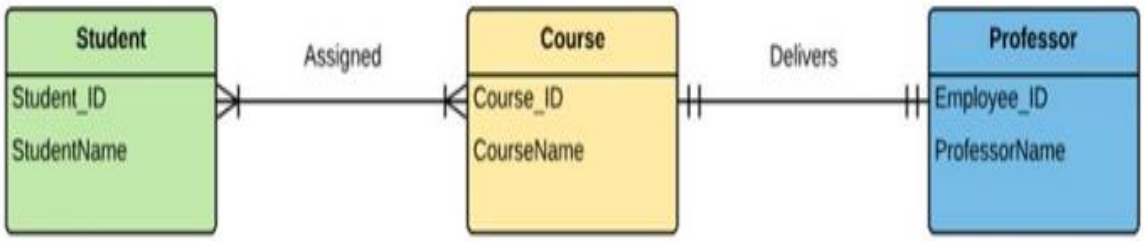
Entity	Primary Key	Attribute
Student	Student_ID	StudentName
Professor	Employee_ID	ProfessorName
Course	Course_ID	CourseName



For Course Entity, attributes could be Duration, Credits, Assignments, etc. For the sake of ease we have considered just one attribute.

Step 5) Create the ERD

A more modern representation of ERD Diagram



Best Practices for Developing Effective ER Diagrams

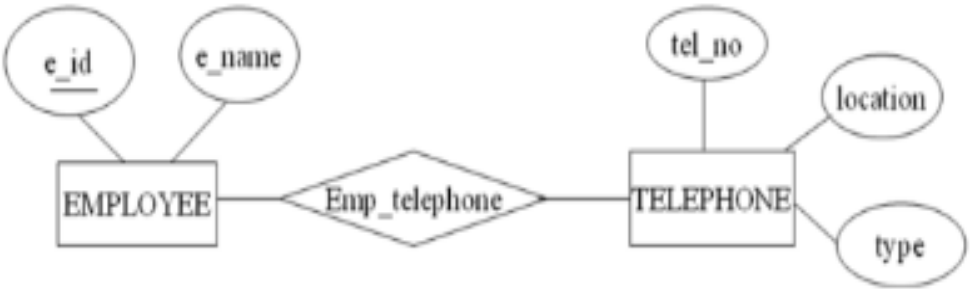
- Eliminate any redundant entities or relationships
- You need to make sure that all your entities and relationships are properly labeled
- There may be various valid approaches to an ER diagram. You need to make sure that the ER diagram supports all the data you need to store
- You should assure that each entity only appears a single time in the ER diagram
- Name every relationship, entity, and attribute are represented on your diagram
- Never connect relationships to each other
- You should use colors to highlight important portions of the ER diagram

Design Issues in Entity-Relationship (ER) Modeling:

1. Entity vs. Attribute
2. Entity vs. Relationship
3. Binary vs. Ternary Relationships
4. Placement of Relationship Attributes

1.Entity vs. Attribute

In the real world situations, sometimes it is difficult to select the property as an attribute or an entity set. The issue of Entity vs. Attribute is about deciding whether a piece of data should be treated as an entity or just an attribute of another entity.

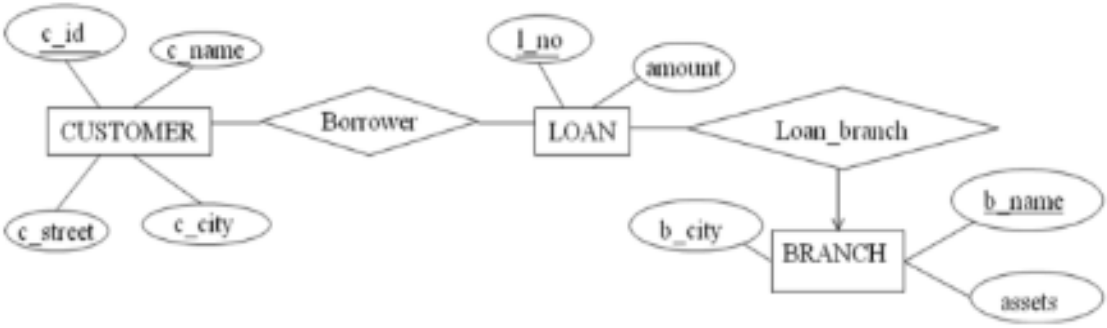


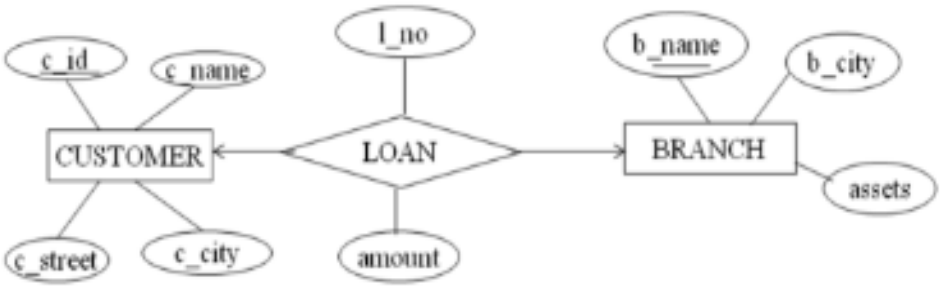
Example:

- Telephone number (tel_no) is shown as an attribute of the Employee entity.
- This works fine if each employee has only one telephone number.
- But if an employee has multiple phone numbers or more details about the phone (like type or location), the telephone number should be treated as a separate entity.

2. Entity sets vs. Relationship sets

Sometimes a real-world concept can be modeled either as an entity or as a relationship, and choosing the right one is important.





The two ER diagrams you provided illustrate Entity vs. Relationship design challenges.

First Diagram (Top):

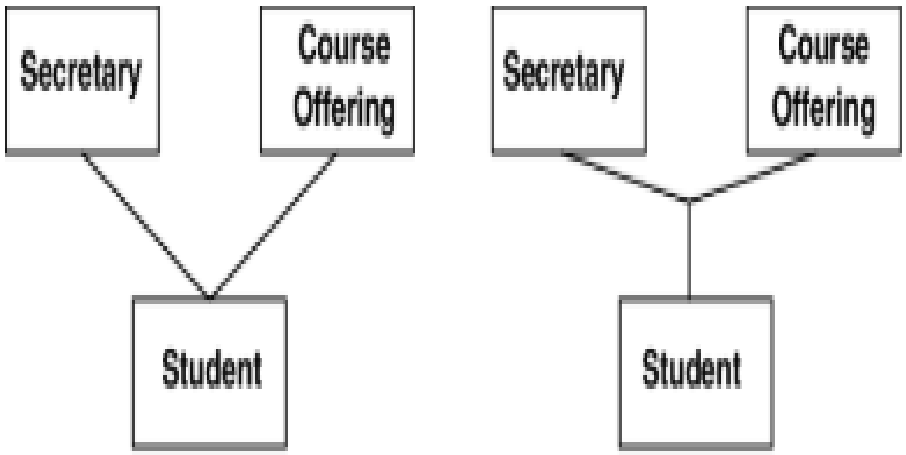
- The relationship Borrower connects the CUSTOMER and LOAN entities.
- The relationship Loan_branch connects the LOAN and BRANCH entities.
- Here, CUSTOMER, LOAN, and BRANCH are represented as separate entities, and relationships are explicitly drawn between them.

Second Diagram (Bottom):

- Similar entities are present: CUSTOMER, LOAN, and BRANCH.
- However, the relationships are not explicitly named or detailed like in the first diagram.
- In the first diagram, relationships are explicitly named (e.g., “Borrower” and “Loan_branch”) and treated as important elements of the model.
- In the second diagram, relationships exist, but their importance or attributes are not detailed.

3.Binary vs. Ternary Relationships

Should a relationship involving three entities be represented as a single ternary relationship or as several binary relationships?



2-a Binary relations 2-b Ternary relation

The decision here is whether to use two separate binary relationships or a single ternary relationship to represent connections between three entities.

First Diagram (Binary Relations):

- Shows two separate relationships:
- “Secretary” is related to “Student.”
- “Course Offering” is related to “Student.”
- This setup treats these relationships independently.

Second Diagram (Ternary Relation):

- Shows one combined relationship between “Secretary,” “Student,” and “Course Offering.”
- Instead of separate relationships, all three entities are connected in one ternary relationship.
- If the relationship among three entities is deeply interdependent, then a ternary relationship is more suitable.
- If the relationships can function independently, it’s better to use binary relationships for clarity.

4. Placement of Relationship Attributes

In an Entity-Relationship (ER) model, where relationship attributes are placed depends on the cardinality ratio — meaning how entities relate to each other in terms of numbers.

1. One-to-Many (1:M)

- If a relationship is one-to-many (like Customer → Places → Order), the attributes should be placed with the “many” side of the relationship.
- Example: If “Order Date” is an attribute of the Places relationship, it should be stored with the Order entity (many side), not with the Customer entity (one side).

2. One-to-One (1:1)

- If a relationship is one-to-one (like Person → Has → Passport), the attributes can be placed on either entity.
- Example: If “Issue Date” is an attribute of Has, it could be stored with Person or Passport, but the decision usually depends on how data retrieval is optimized.

3. Many-to-Many (M:N)

- When a relationship is many-to-many (like Student → Enrolled In → Course), attributes cannot be stored in either entity.
- Instead, a new entity is created for the relationship.
- Example: If “Enrollment Date” is an attribute of Enrolled In, it should belong to an independent Enrollment entity, not the Student or Course entity.

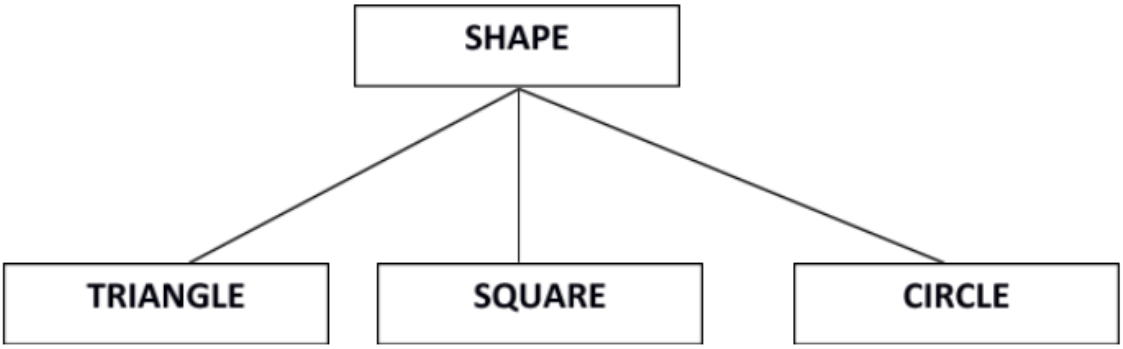
Extended Entity-Relationship (EE-R) Model

EER is a high-level data model that incorporates the extensions to the original **ER model**. Enhanced ERD are high level models that represent the requirements and complexities of complex database.

These concepts are used to create EE-R diagrams.

Subclasses and Super class

Super class is an entity that can be divided into further subtype.
For **example** ? consider Shape super class.

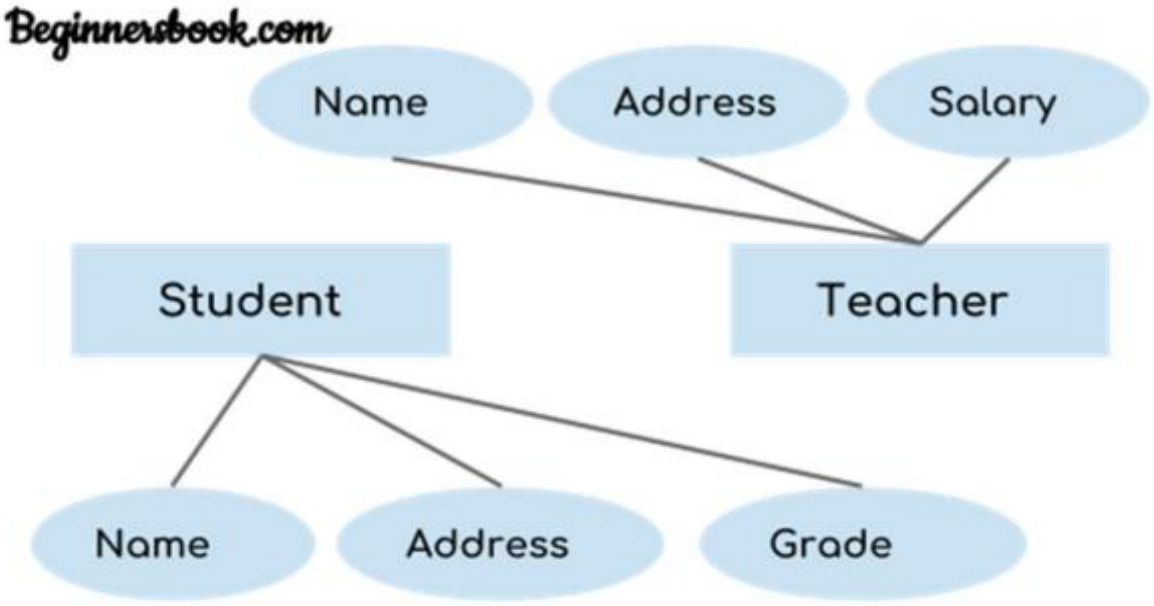


Super class shape has sub groups: Triangle, Square and Circle.
Sub classes are the group of entities with some unique attributes. Sub class inherits the properties and attributes from super class.

Extended ER features -

Generalization

Generalization is a process in which the common attributes of more than one entities form a new entity. This newly formed entity is called generalized entity.
Generalization Example
Lets say we have two entities Student and Teacher. Attributes of Entity Student are: Name, Address & Grade Attributes of Entity Teacher are: Name, Address & Salary
The ER diagram before generalization looks like this:

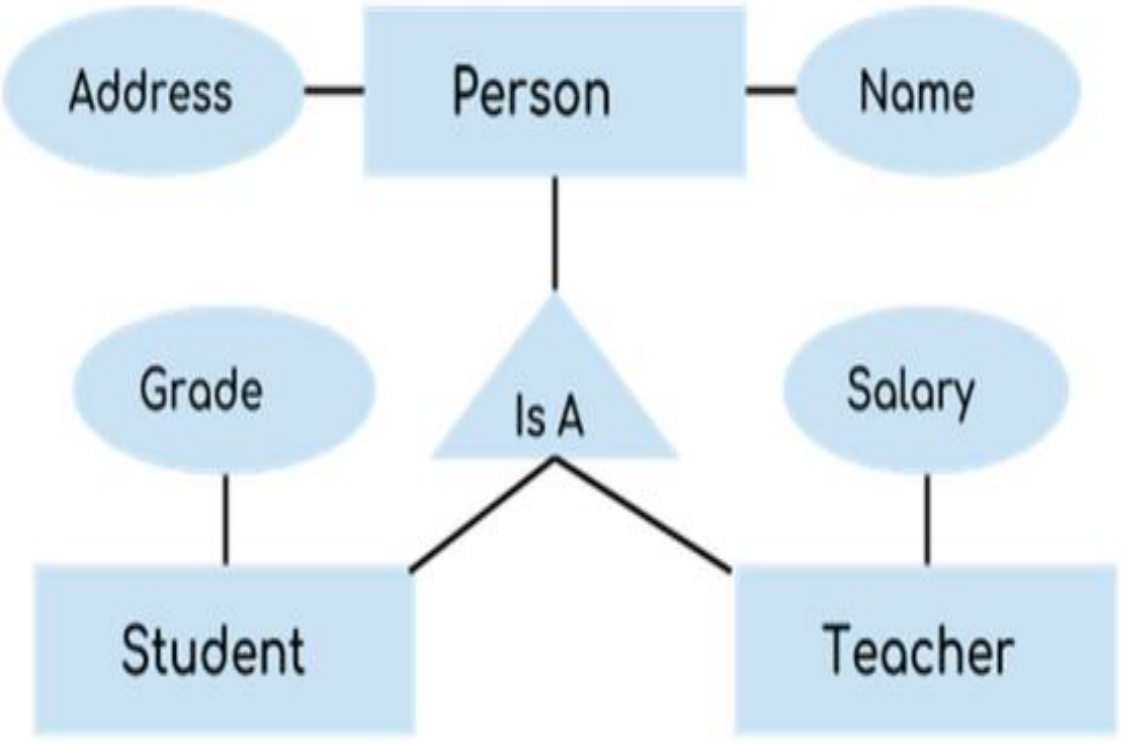


Before Generalization

These two entities have two common attributes: Name and Address, we can make a generalized entity with these common attributes. Lets have a look at the ER model after generalization.

The ER diagram after generalization:

We have created a new generalized entity Person and this entity has the common attributes of both the entities. As you can see in the following ER diagram that after the generalization process the entities Student and Teacher only has the specialized attributes Grade and Salary respectively and their common attributes (Name & Address) are now associated with a new entity Person which is in the relationship with both the entities (Student & Teacher).



Generalization

Note:

1. Generalization uses bottom-up approach where two or more lower level entities combine together to form a higher level new entity.
2. The new generalized entity can further combine together with lower level entity to create a further higher level generalized entity.

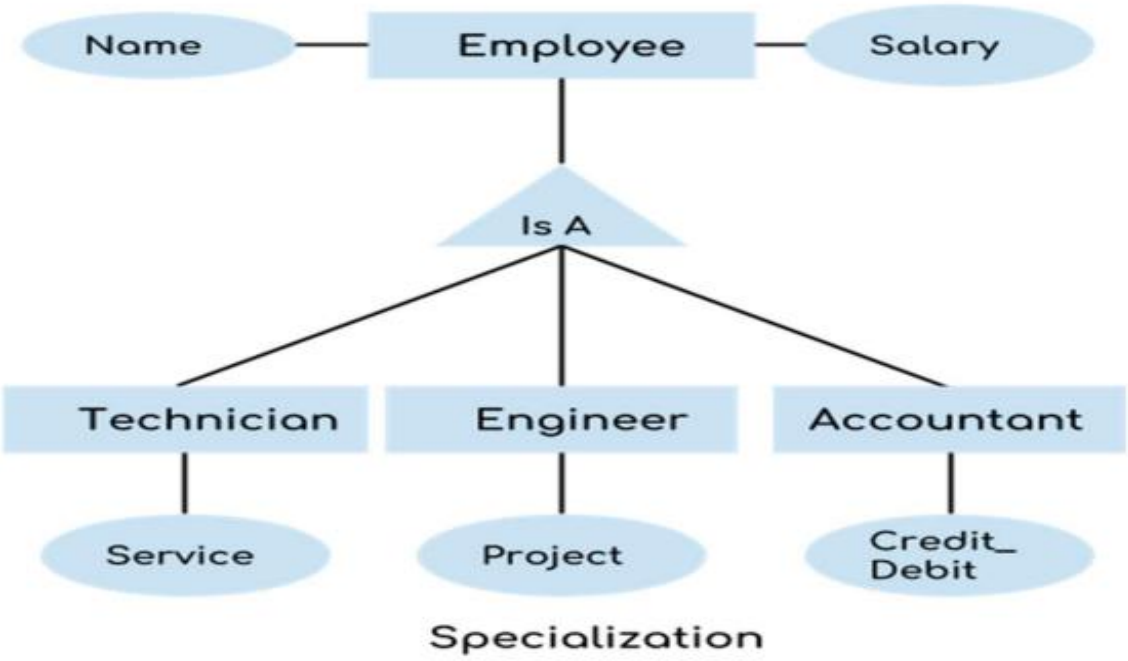
Specialization

Specialization is a process in which an entity is divided into sub-entities. You can think of it as a reverse process of generalization, in generalization two entities combine together to form a new higher level entity. Specialization is a top-down process.

The idea behind Specialization is to find the subsets of entities that have few distinguish attributes. For example – Consider an entity employee which can be further classified as subentities Technician, Engineer & Accountant because these sub entities have some distinguish attributes.

Specialization Example

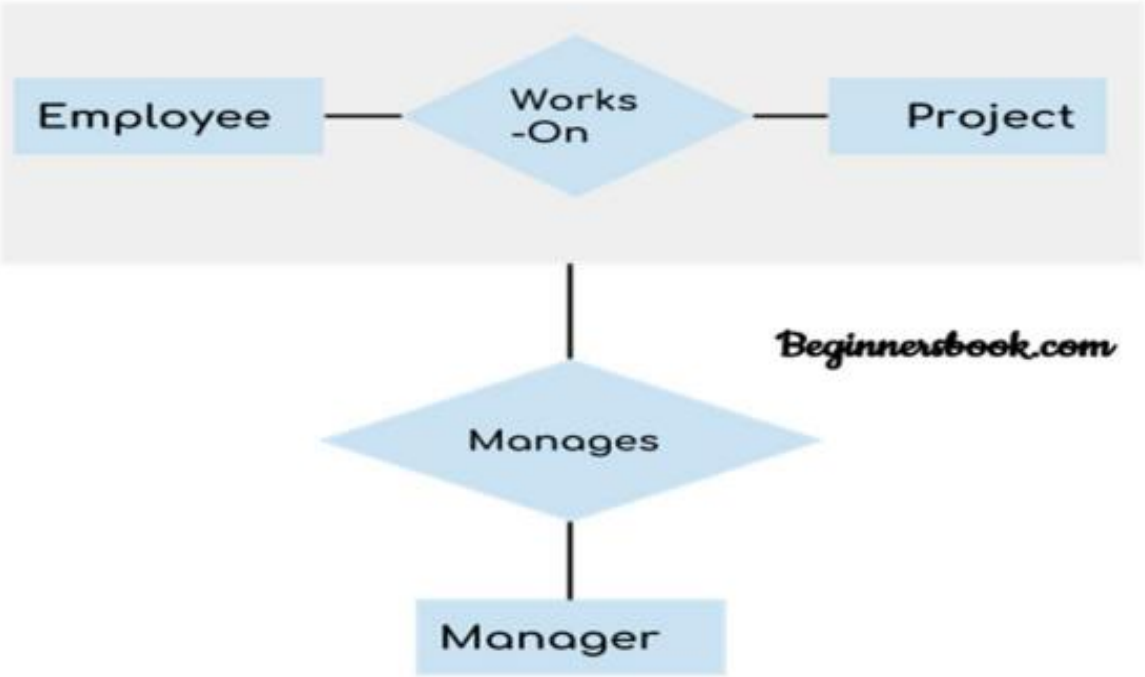
In this diagram, we can see that we have a higher level entity “Employee” which we have divided in sub entities “Technician”, “Engineer” & “Accountant”. All of these are just an employee of a company, however their role is completely different and they have few different attributes. Just for the example, I have shown that Technician handles service requests, Engineer works on a project and Accountant handles the credit & debit details. All of these three employee types have few attributes common such as name & salary which we had left associated with the parent entity “Employee” as shown in the above diagram.



Aggregation

Aggregation is a process in which a single entity alone is not able to make sense in a relationship so the relationship of two entities acts as one entity. I know it sounds confusing but don't worry the example we will take, will clear all the doubts.

Aggregation Example



In real world, we know that a manager not only manages the employee working under them but he has to manage the project as well. In such scenario if entity "Manager" makes a "manages" relationship with either "Employee" or "Project" entity alone then it will not make any sense because he has to manage both. In these cases the relationship of two entities acts as one entity. In our example, the relationship "Works-On" between "Employee" & "Project" acts as one entity that has a relationship "Manages" with the entity "Manager".

Converting ER/EER diagrams to relational tables-

After designing an **ER Diagram**,

- ER diagram is converted into the tables in relational model.
- This is because relational models can be easily implemented by RDBMS like MySQL , Oracle etc.

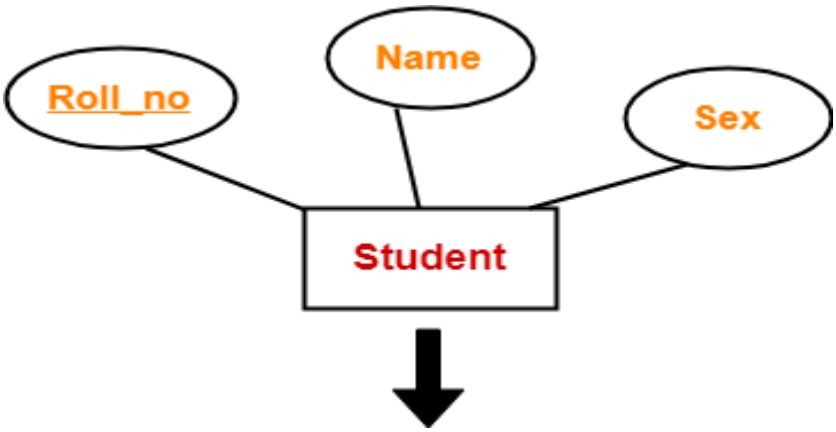
Following rules are used for converting an ER diagram into the tables-

Rule-01: For Strong Entity Set With Only Simple Attributes-

A strong entity set with only simple attributes will require only one table in relational model.

- Attributes of the table will be the attributes of the entity set.
- The primary key of the table will be the key attribute of the entity set.

Example-



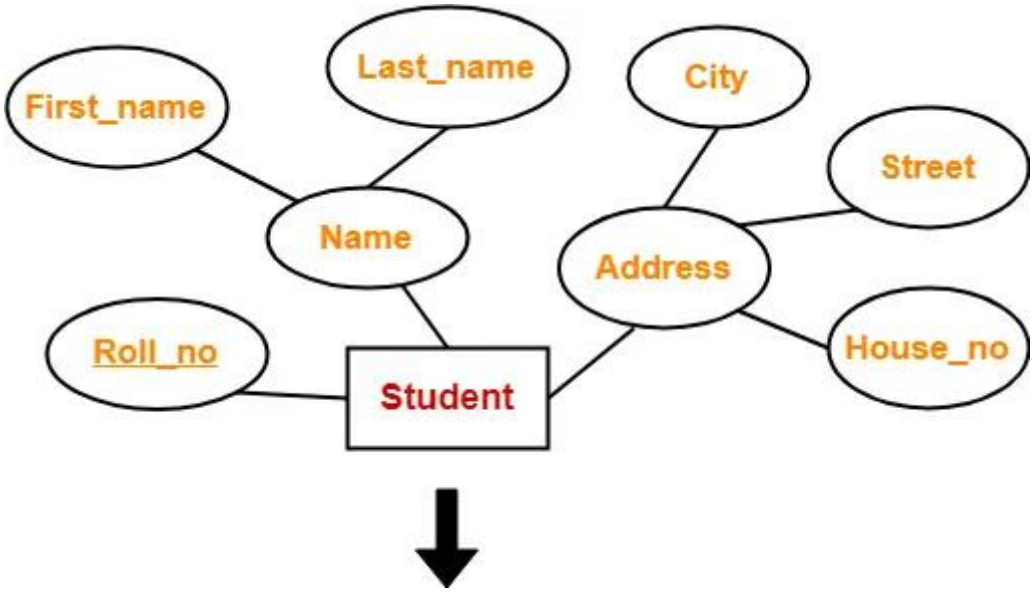
Roll_no	Name	Sex

Schema : Student (Roll_no , Name , Sex)

Rule-02: For Strong Entity Set With Composite Attributes-

- A strong entity set with any number of composite attributes will require only one table in relational model.
- While conversion, simple attributes of the composite attributes are taken into account and not the composite attribute itself.

Example-



Roll_no	First_name	Last_name	House_no	Street	City

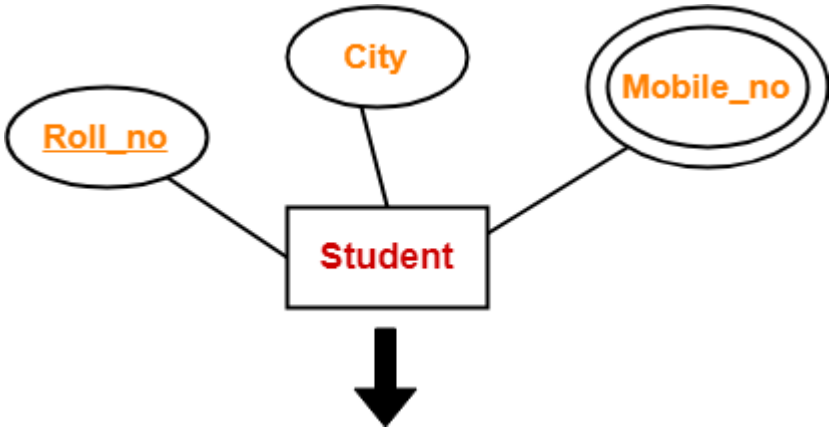
Schema : Student (Roll_no , First_name , Last_name , House_no , Street , City)

Rule-03: For Strong Entity Set With Multi Valued Attributes-

A strong entity set with any number of multi valued attributes will require two tables in relational model.

- One table will contain all the simple attributes with the primary key.
- Other table will contain the primary key and all the multi valued attributes.

Example-



Roll_no	City

Roll_no	Mobile_no

Rule-04: Translating Relationship Set into a Table-

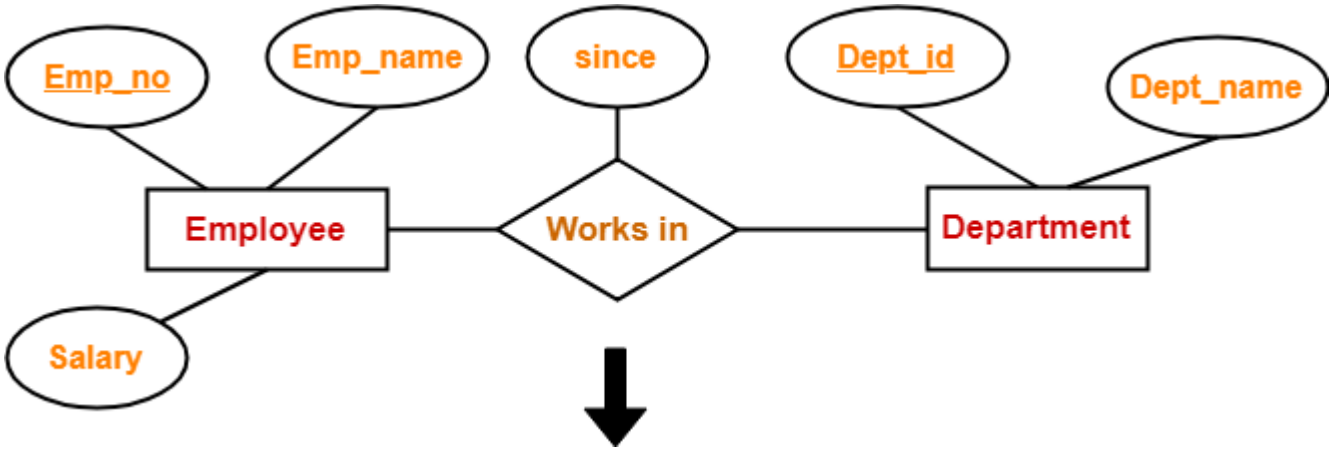
A relationship set will require one table in the relational model.

Attributes of the table are-

- Primary key attributes of the participating entity sets
- Its own descriptive attributes if any.

Set of non-descriptive attributes will be the primary key.

Example-



Emp_no	Dept_id	since

Schema : Works in (Emp_no , Dept_id , since)

NOTE-

If we consider the overall ER diagram, three tables will be required in relational model-

- One table for the entity set “Employee”
- One table for the entity set “Department”
- One table for the relationship set “Works in”

Rule-05: For Binary Relationships With Cardinality Ratios-

The following four cases are possible-

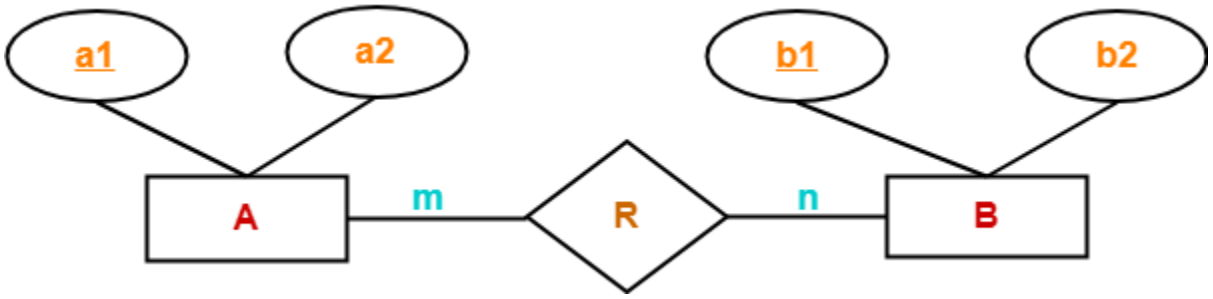
Case-01: Binary relationship with cardinality ratio m:n

Case-02: Binary relationship with cardinality ratio 1:n

Case-03: Binary relationship with cardinality ratio m:1

Case-04: Binary relationship with cardinality ratio 1:1

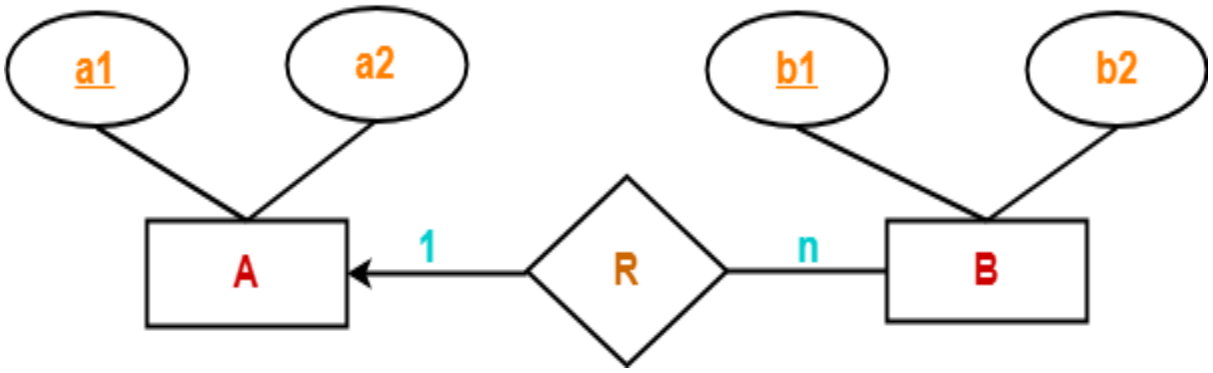
Case-01: For Binary Relationship With Cardinality Ratio m:n



Here, three tables will be required-

1. A (a1 , a2)
2. R (a1 , b1)
3. B (b1 , b2)

Case-02: For Binary Relationship With Cardinality Ratio 1:n

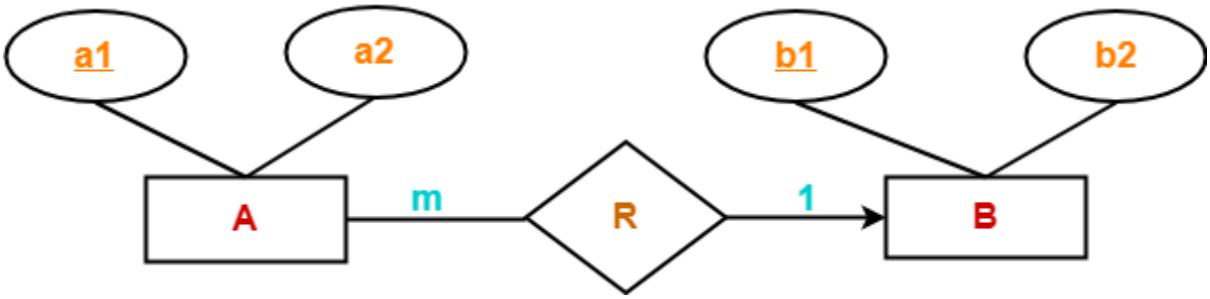


Here, two tables will be required-

1. A (a1 , a2)
2. BR (a1 , b1 , b2)

NOTE- Here, combined table will be drawn for the entity set B and relationship set R.

Case-03: For Binary Relationship With Cardinality Ratio m:1

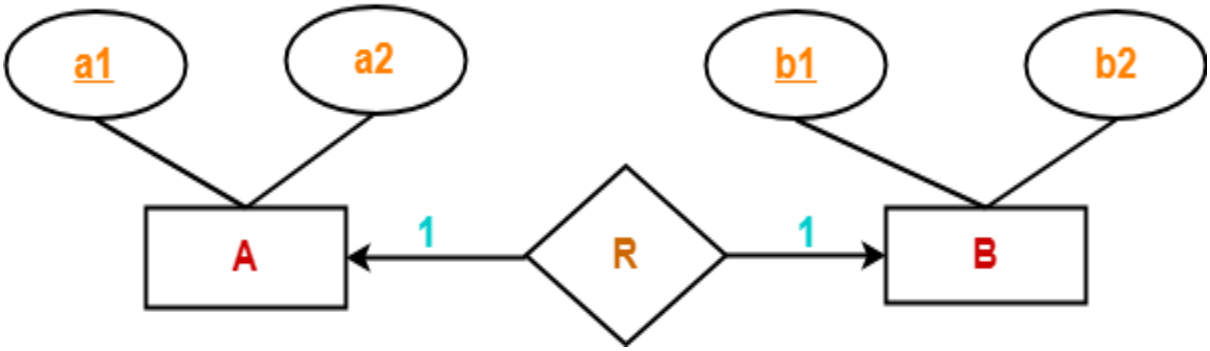


Here, two tables will be required-

1. AR (a1 , a2 , b1)
2. B (b1 , b2)

NOTE- Here, combined table will be drawn for the entity set A and relationship set R.

Case-04: For Binary Relationship With Cardinality Ratio 1:1



Here, two tables will be required. Either combine 'R' with 'A' or 'B'

Way-01:

1. AR (a1 , a2 , b1)
2. B (b1 , b2)

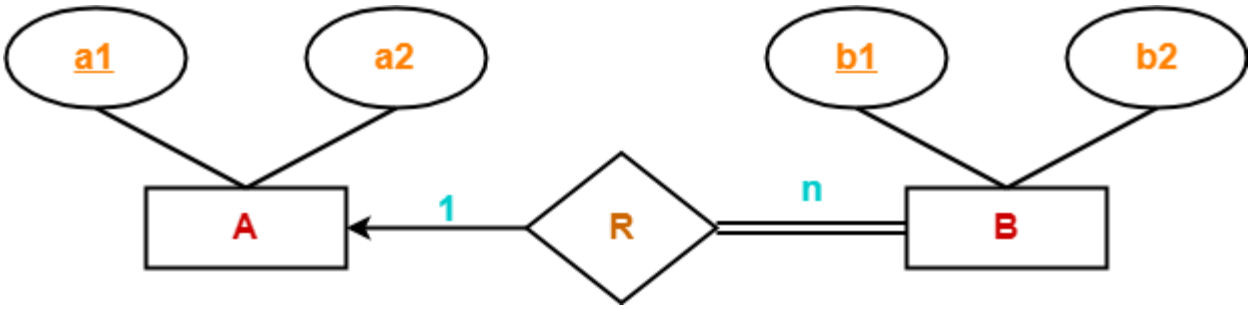
Way-02:

1. A (a1 , a2)
2. BR (a1 , b1 , b2)

Rule-06: For Binary Relationship With Both Cardinality Constraints and Participation Constraints-

- Cardinality constraints will be implemented as discussed in Rule-05.
- Because of the total participation constraint, foreign key acquires **NOT NULL** constraint i.e. now foreign key can not be null.

Case-01: For Binary Relationship With Cardinality Constraint and Total Participation Constraint From One Side-



Because cardinality ratio = 1 : n , so we will combine the entity set B and relationship set R.

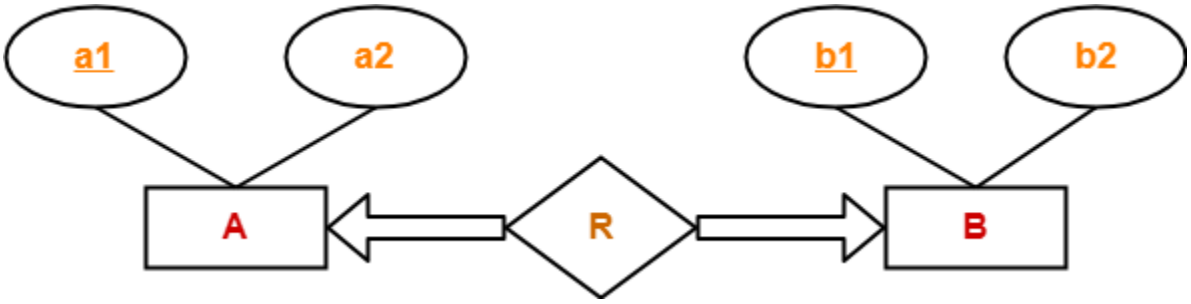
Then, two tables will be required-

1. A (a1 , a2)
2. BR (a1 , b1 , b2)

Because of total participation, foreign key a1 has acquired NOT NULL constraint, so it can't be null now.

Case-02: For Binary Relationship With Cardinality Constraint and Total Participation Constraint From Both Sides-

If there is a key constraint from both the sides of an entity set with total participation, then that binary relationship is represented using only single table.

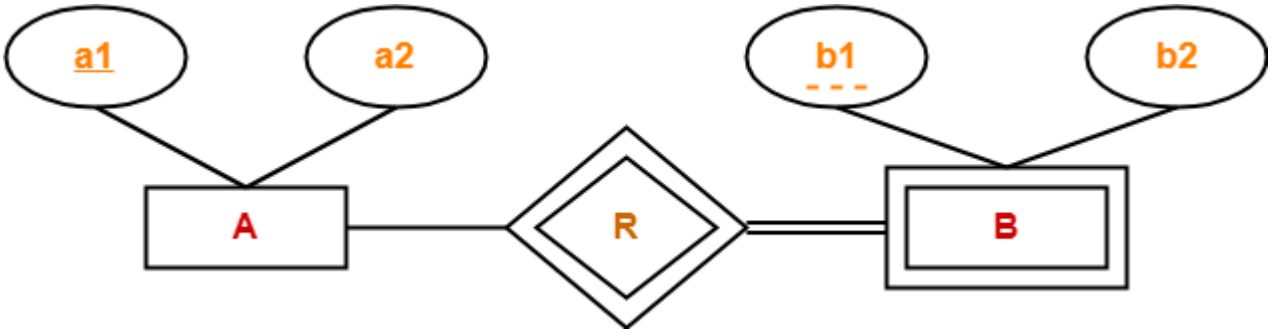


Here, Only one table is required.

- ARB (a1 , a2 , b1 , b2)

Rule-07: For Binary Relationship With Weak Entity Set-

Weak entity set always appears in association with identifying relationship with total participation constraint.



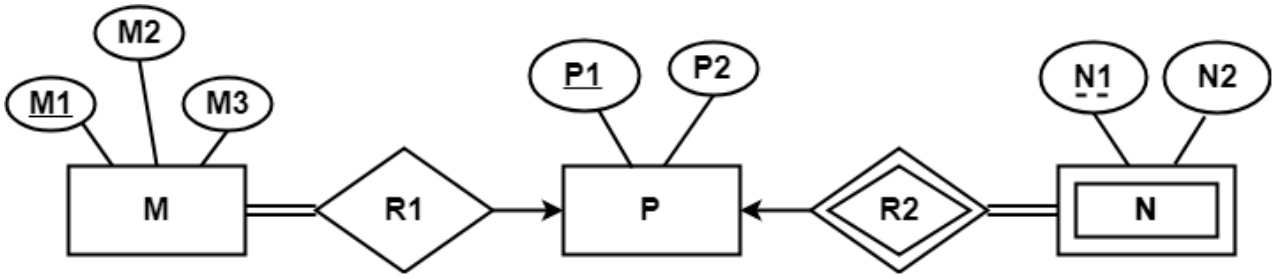
Here, two tables will be required-

1. A (a1 , a2)
2. BR (a1 , b1 , b2)

PROBLEMS BASED ON CONVERTING ER DIAGRAM TO TABLES-

Problem-01:

Find the minimum number of tables required for the following ER diagram in relational model-



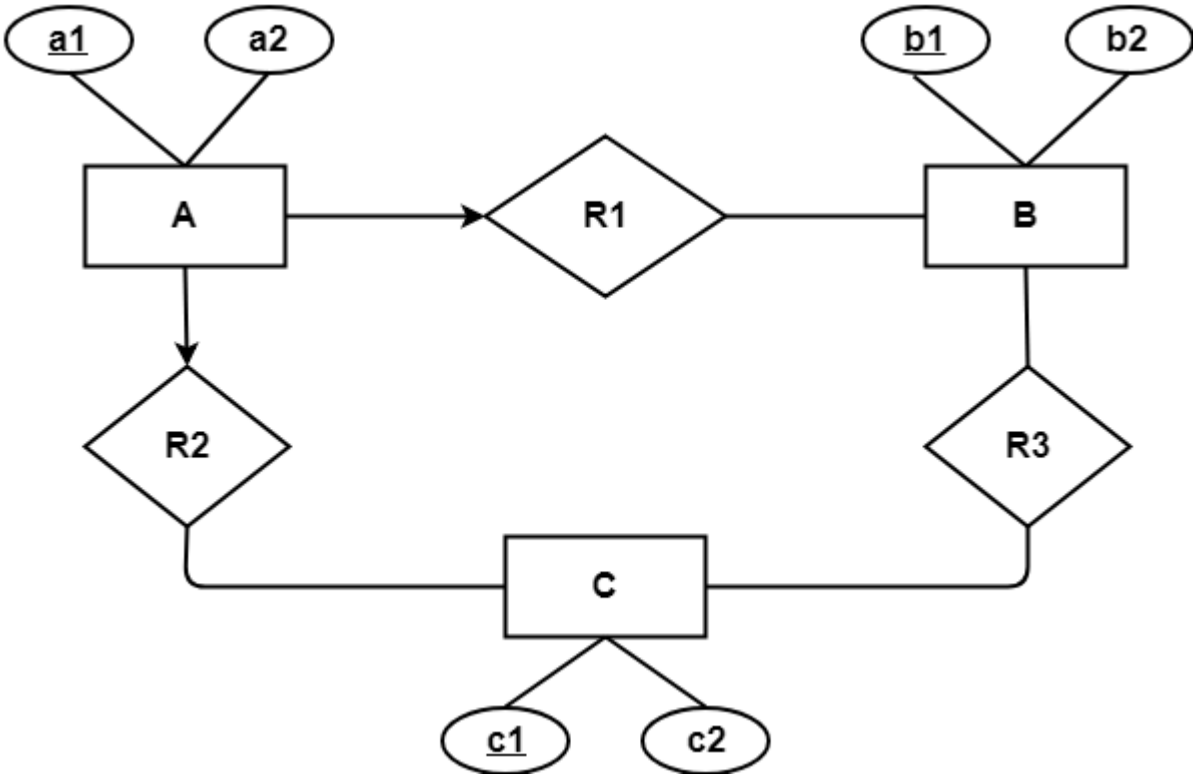
Solution-

Applying the rules, minimum 3 tables will be required-

- MR1 (M1 , M2 , M3 , P1)
- P (P1 , P2)
- NR2 (P1 , N1 , N2)

Problem-02:

Find the minimum number of tables required to represent the given ER diagram in relational model-



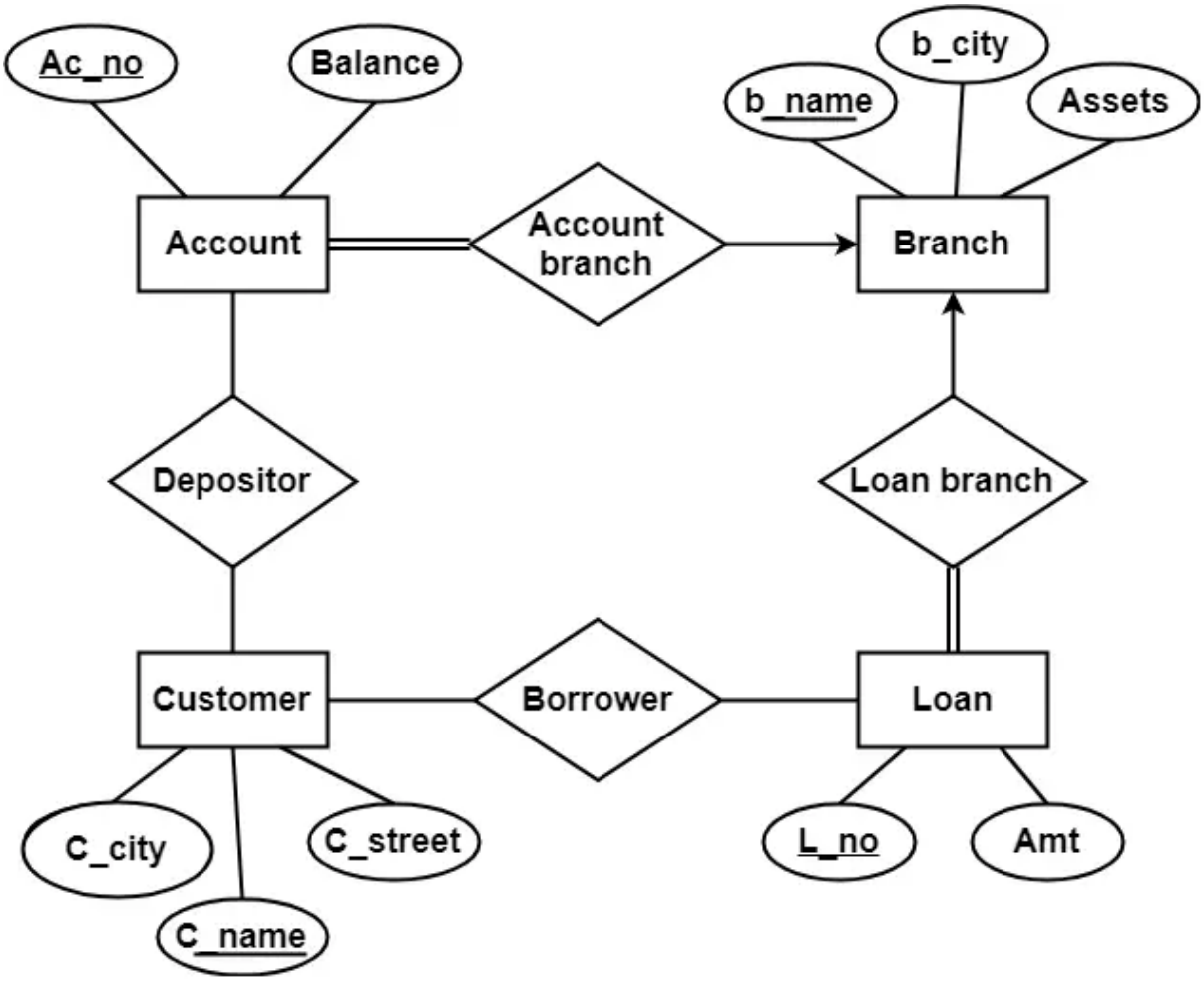
Solution-

Applying the rules, minimum 4 tables will be required-

- AR1R2 (a1 , a2 , b1 , c1)
- B (b1 , b2)
- C (c1 , c2)
- R3 (b1 , c1)

Problem-03:

Find the minimum number of tables required to represent the given ER diagram in relational model-



Solution-

Applying the rules that we have learnt, minimum 6 tables will be required-

- Account (Ac_no , Balance , b_name)
- Branch (b_name , b_city , Assets)
- Loan (L_no , Amt , b_name)
- Borrower (C_name , L_no)
- Customer (C_name , C_street , C_city)
- Depositor (C_name , Ac_no)